

LightLink: Enhancing Smartwatch Sensing and Actuation Through Visible Light Communication

Antony Albert Raj Irudayaraj

Department of Computer Science,
University of Calgary
Calgary, Canada
antony.irudayaraj@ucalgary.ca

Aditya Shekhar Nittala

Department of Computer Science,
University of Calgary
Calgary, Canada
anittala@ucalgary.ca



Figure 1: LightLink enables adding different sensor and actuator add-ons to a smartwatch: (a) shows a person blowing into a bezel mounted alcohol sensor for on-demand breath alcohol level measurement; (b) four button user interface attached to the watch strap for eyes-free interaction; (c) Tilt switch sensor band attached to the forearm to track reps count in exercises; (d) Vibration band attached to the forearm to deliver haptic patterns; (e) LED display add-on attached to the watch strap for quicker and easier viewing of notifications/navigation instructions.

ABSTRACT

Smartwatches are equipped with fixed sensors (e.g., ECG, IMUs) or actuators designed for a broad audience. However, some users need specialized smartwatch functionality, creating a demand for add-on devices that extend a smartwatch's capabilities. Our work explores the design of add-on devices that communicate with the smartwatch using visible light. To demonstrate the feasibility of our approach, we conduct a series of characterization experiments to assess the ability of the smartwatch's ambient light sensor and screen as a light-based receiver and transmitter, respectively, and develop strategies for establishing a reliable communication link. Our approach is low-cost, has minimal instrumentation of a smartwatch, and supports the addition of input and output capabilities. To validate our approach, we design and test a suite of add-ons that expand the sensing (e.g. buttons, body temperature, etc.) and output (visual feedback and haptic wristband) capabilities of the smartwatch. We also contribute a toolkit framework comprising our hardware designs and a software library that enables designers and developers to quickly create applications that integrate add-ons with smartwatches using our approach.

CCS CONCEPTS

• Human-centered computing → Ubiquitous and mobile devices; • Hardware → Sensors and actuators.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

UIST '25, September 28–October 1, 2025, Busan, Republic of Korea

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-2037-6/2025/09

<https://doi.org/10.1145/3746059.3747623>

KEYWORDS

Smartwatch sensor and actuator add-ons, Visible Light Communication, Wearable devices

ACM Reference Format:

Antony Albert Raj Irudayaraj and Aditya Shekhar Nittala. 2025. LightLink: Enhancing Smartwatch Sensing and Actuation Through Visible Light Communication. In *The 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*, September 28–October 1, 2025, Busan, Republic of Korea. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3746059.3747623>

1 INTRODUCTION

Smartwatches are essential for daily life, providing quick access to notifications, fitness metrics, and reminders. They feature various sensors (e.g., IMU, ECG, ambient light) for fitness tracking, health monitoring, and adaptive screen brightness control. Additionally, they include output interfaces like displays, speakers, and vibration motors to communicate with users. Commercial smartwatches typically have a fixed set of sensors and output devices to cater to a wide audience. Augmenting them with additional sensors and output devices opens up new applications for niche audiences. For example, a strain sensor could assist in patient rehabilitation, or a body temperature sensor could help monitor fever. Furthermore, integrating more advanced sensors, such as UV or air quality monitors and future technologies like non-invasive sweat glucose sensors, into smartwatches can be challenging. However, it may be more feasible to offer them as add-on devices. This approach allows for easier customization, enables repeated use (for example, by swapping out saturated glucose sensing strips), and reduces integration costs. The challenge is adding these sensors/actuators to the watch and establishing a communication link in a low-cost way with minimal augmentation to enable rapid prototyping.

Using single-function devices with specialized sensors and dedicated processors is (1) expensive, (2) requires hardware, firmware, and software expertise, and (3) is unlikely to be incorporated into mass-produced smartwatches due to increased costs and complexity. This presents an opportunity for niche add-on sensors or actuators that enhance a smartwatch's usefulness for specific needs. Cost-effective add-on interfaces can leverage a smartwatch's built-in sensors to establish a communication link with it.

Prior HCI research has explored various ways to augment a smartwatch with additional capabilities [17, 28, 51]. However, these typically require dedicated hardware and substantial instrumentation to the smartwatch. Additionally, they lack modularity and do not offer the flexibility to easily add/remove various functionalities. Then, another stream of work focuses on adding input capabilities by leveraging built-in infrastructure on devices like a microphone and touchscreen [22, 47]. However, these are limited to only touch and non-voice speech input.

More recently and highly relevant to our work, WatchLink [65] explored using a built-in ECG sensor for communicating from an external sensor to the watch. However, this approach is limited by several factors: it only supports unidirectional communication, has limited sensing resolution, and can only support input/sensor add-ons, but not output add-ons such as LEDs or haptic feedback. Moreover, it is computationally intensive, requires FFT computation, and real-time measurements are not possible due to API restrictions preventing access to raw ECG signals on most smartwatches, as reported in the publication [65].

To overcome these challenges and enable bidirectional communication with minimal instrumentation, we explore using modulated visible light signals to establish a bidirectional communication link between a smartwatch and peripheral add-on devices, using the watch's built-in light sensor and screen as a light receiver and transmitter, respectively. This approach enables integrating both input and output devices to the smartwatch, minimal computation for interpreting sensed values, real-time data display, and seamless integration into custom applications.

This paper makes the following contributions:

- We contribute the design of a low-cost approach for augmenting a smartwatch with additional sensing and actuation capabilities using visible light communication. While visible light communication has been extensively used for diverse applications, **to the best of our knowledge, for the first time, we explore visible light communication in a compact form factor for wearable devices.**
- Our work characterizes the smartwatch's ambient light sensor (ALS) and screen as a light receiver and transmitter respectively, through a series of technical experiments.
- We contribute a plug-and-play modular toolkit that enables augmenting a smartwatch with diverse input and output capabilities. While physical prototyping toolkits are gaining widespread attention in HCI [16, 25, 27], to the best of our knowledge, we are yet to see toolkits for augmenting smartwatch capabilities.
- We present an application-driven validation by developing eight plug-and-play modular add-on interfaces for smartwatches, encompassing a wide range of sensors and actuators. Through these functional and interactive add-ons, we demonstrate that

LightLink supports digital (Figure 1b,c) and analog sensors (Figure 1a), infrared (IR) transmission, haptic interface (Figure 1d) and visual output (Figure 1e). To the best of our knowledge, no existing toolkit or augmentation system for smartwatches offers such diverse functionality.

- We contribute results from a series of technical experiments to evaluate LightLink's data transmission characteristics under various external factors.

2 BACKGROUND AND RELATED WORK

Our work falls at the intersection of three main areas: (1) augmenting smartwatches with additional functionalities, (2) communication mechanisms with wearables, and (3) toolkit research for physical computing.

2.1 Augmenting Smartwatches with Additional Functionalities

Even since the proliferation of smartwatches, the HCI community has been actively exploring various techniques for expanding the input and output capabilities of smartwatches [51]. Prior research focused on adding additional hardware to smartwatches for addressing the fat-finger problem [31, 69, 70, 76, 78], for additional sensing capabilities [14, 39, 51, 58, 75, 76], for using the strap as an interactive area [28, 51], to augment additional display capabilities [28, 43, 45, 68], and for additional tactile feedback capabilities [13, 18, 62]. Then, another stream of work has explored the use of existing sensors and functionalities of smartwatches for novel interactions [9, 26, 32, 65]. For instance, prior work expanded the sensing capabilities of smartwatches by overclocking the in-built IMUs [30, 32]. Detachable Watch [26] and WatchPen [19] detached the watch body from the wristband to increase the functionality to enable an ecosystem of sensor-laden bands and cross-device interaction. More recently, WatchLink explored the ECG channel for augmenting sensor add-ons to smartwatches [65]. While impressive, using the ECG channel can only add input/sensing capabilities but not output (visual/tactile) capabilities.

LightLink aims to expand the sensing capabilities of smartwatches in a cost-effective that does not compromise the watch's original functionality. To enhance versatility, we explore add-on sensors and the potential for augmenting output capabilities, such as visual and tactile feedback. Our work is inspired by WatchLink [65]; however, by utilizing visible light communication through the ambient light sensor (ALS) and watch screen, we can enhance smartwatches with both sensor add-ons and output features (visual and tactile).

2.2 Communication Mechanisms with Wearables

This section reviews the most commonly used communication mechanisms explored for augmenting smartwatch capabilities..

2.2.1 Radio Communication. Radio communication is essential for connecting wearable devices like smartwatches and earbuds to mobile phones, utilizing technologies such as WiFi, NFC, and BLE for different purposes. WiFi connects devices to the internet, Bluetooth pairs personal devices, and NFC facilitates communication with

Related Work	Medium	Flow	Instrumentation	Input		Output	
				Discrete	Continuous	Visual	Haptic
WatchLink [65]	ECG	Unidirectional	Minimal	✓	✓	✗	✗
Wristband.IO [51]	BLE	Bidirectional	Dedicated Hardware	✓	✓	✓	✗
Watch+Strap [28]	Wifi	Bidirectional	Dedicated Hardware	✓	✓	✓	✗
WatchConnect [17]	USB Serial	Bidirectional	Dedicated Hardware	✓	✓	✓	✗
Magnetips [39]	USB Serial	Bidirectional	Dedicated Hardware	✓	✓	✓	✓
ScatterWatch [45]	n/a	n/a	Dedicated Hardware	✓	✗	✓	✗
CaseTouch [60]	BLE	Unidirectional	Minimal	✓	✓	✗	✗
Takahashi et al. [62]	BLE	Bidirectional	Dedicated Hardware	✗	✗	✗	✓
ShiftTouch [22]	passive touch	Unidirectional	Minimal	✓	✓	✗	✗
Whoosh [47]	microphone	Unidirectional	Minimal	✓	✓	✗	✗
This work	Visible Light	Bidirectional	Minimal	✓	✓	✓	✓

Table 1: Comparison of Related work that extended the interactive capabilities of smartwatches. They either use minimal hardware (passive touch sensors) for expanding the touch input area [22, 39, 60] or dedicated hardware with Wifi/BLE or USB serial communication for expanding the display capabilities and notifications [28, 51]. WatchLink [65], used inbuilt ECG sensors for augmenting smartwatches with sensor add-ons, but the technique is limited to augmenting input capabilities.

nearby devices. NFC-enabled smartphones can power and update battery-free devices [11, 12, 77]. However, smartwatches' NFC capabilities are limited to tags and cannot operate as a reader, limiting their use for communication with external devices. WiFi consumes significant power (100 mW) while BLE is more energy-efficient (~5 mW) [65]. Unlike BLE or WiFi, reporting a single power consumption value for LightLink is not trivial, as its power usage varies dynamically with the sensor output. In typical wireless systems, the sensor value is first read by a microcontroller and then transmitted via a communication module, resulting in a more consistent power profile. In contrast, LightLink tightly integrates sensing and communication, with the LED current directly modulated based on the sensor value. This coupling leads to variable power consumption. The power consumption range of LightLink sensor add-on prototypes evaluated in this work ranged from 7.6 mW to 115 mW at nominal operating conditions. Note, these values include the power consumed by both the sensor and the communication circuitry.

Radio systems often require pairing, costly hardware, complex firmware for managing the radio stack, large transceiver chips, intricate antenna, and circuit designs. Light Link offers a simpler, lightweight communication solution using LEDs and smartwatch screens, streamlining the development and integration of add-ons for wearable applications.

2.2.2 Using Built-in Sensors and Actuators for Communication. Although dedicated radio modules are integrated into smartwatches and phones for communication, prior research has investigated leveraging built-in infrastructure on wearable and mobile devices to create novel communication channels without altering the device hardware.

2.2.2.1 Acoustic Communication: Acoustic data transmission leverages speakers and microphones in smartwatches and smartphones to enable communication [8, 41, 42]. To achieve this, the transmitting device sends a modulated sound wave decoded by the receiver's microphone to interpret the information. Another

approach uses a passive tubular structure as a casing to send non-voice acoustic inputs to the smartwatch [47]. However, using audio for communication is susceptible to ambient noise.

2.2.2.2 Using Touchscreen: Alternatively, touch screens on mobile devices enable communication with an external device. Passive objects embedded with conductive ink [6, 29, 52–54] transmit static information to a phone. While these passive devices typically convey fixed data, previous work used passive touch electrodes to reduce screen occlusion and augment the touch surface beyond the screen of the watch or phone [22, 60]. Glucose sensor readings from an external device are communicated to the phone via its touchscreen [66]. In addition to capacitive techniques, prior work utilized magnetic hall sensor grids to identify objects [37] and detect their posture above a screen [36]. While touch input on a smartwatch can also be used for communication, it may not be ideal due to the limited display area.

2.2.2.3 Using Structured Vibrations: Information is encoded into vibration patterns using the motors, while an IMU detects and decodes the vibrations to interpret the transmitted data. This method enables communication between multiple mobile phones [20, 61] or between a phone and an external vibration transmitter [48, 71] or to encode braille [10, 57]. The IMUs in smartwatches [32] or smartphones [3] can sense subtle vibration profiles generated by objects in contact for context-aware applications. Although these systems are low-cost, they are susceptible to noise and require a physical medium, such as a desk or skin, for the signal to propagate.

2.2.2.4 Using Built-In Sensors: In addition to IMUs, microphones and vibrations, prior work also explored the use of other built-in capabilities of mobile devices. For instance, the LiDAR in smartphone cameras were used for laser speckle imaging [4] and for identifying fluids [5]. More related to our work, WatchLink [65] uses the ECG interface on smartwatches to communicate with external sensors attached to it. While impressive, the system only enables a one-way communication link, which limits its capability to input sensor add-ons. Additionally, current operating systems on

smartwatches restrict direct access to ECG waveform, restricting the possibility of real-time sensing and integration into a custom app for the add-on.

2.3 Visible Light Communication with Mobile Devices

Visible Light Communication (VLC) uses light, such as LEDs or displays, to transmit data. The light is modulated to encode information, which is then captured and decoded by a photoreceptor like a photodiode or camera. VLC requires a clear line-of-sight between the transmitter and receiver, making it useful in environments where radio signals might cause interference or are restricted. For more details about this mechanism, we direct the reader to a detailed review by Ismaili et al. [24].

Prior work has explored VLC for transmitting data between two mobile devices. The display of a mobile phone can function as a transmitter, while another mobile device camera acts as a receiver [59, 74]. Similarly, HiLight [35] encodes data by modulating the transparency levels in the displayed image, which is demodulated by the receiving phone's camera. Other systems used multiple transmitters where multiple mobile devices flash color-coded light sequences [55, 73], which are decoded by a centrally positioned camera. VLC was also used for transmitting information through general-purpose LEDs where data was encoded through flashing pulses [2, 7, 23, 50, 72]. More relevant to our approach, prior work used ultrashort light pulses imperceptible to the human eyes [63] to communicate with a receiver or for indoor positioning [38]. Ambient Light Sensors (ALS) present in mobile and wearable devices have been used for VLC. To overcome the slow sampling rate of ALS, Yong-hyeon et al. [21] use time-shifted light intensity modulation to enable communication between an LED and a smartphone's ambient light sensor (ALS) [21]. Similarly, ALS-P [67] adjusts the ALS sampling rate dynamically to generate low-frequency signals by exploiting the frequency aliasing property. While visible light communication has been explored in diverse contexts, to the best of our knowledge, it has yet to be leveraged for augmenting the interactive capabilities of wearables.

2.4 Prototyping Toolkits for Mobile and Wearable Devices

Several toolkits and programming interfaces have been developed recently to enable practitioners and researchers to prototype interactive physical devices [16, 17]. Phidgets is one of the earliest toolkits developed to ease the development of physical computing devices [16]. Similarly, prototyping toolkits have been developed for multi-surface and mobile environments [34, 56]. WatchConnect contributed a toolkit for prototyping cross-device interactions [17]. Similarly, prior work has contributed gesture recognition by oversampling accelerometers in smartwatches [32, 64]. However, practitioners must have deep programming expertise to implement their systems. Recent work has also contributed open-source design to facilitate the construction of VR-enabled wearables [40, 46]. While these systems and toolkits enable novel wearable interactions, they predominantly focus on cross-device interactions [17], expand sensing capabilities [32, 64], or stream data to external applications such as VR apps [40, 46]. However, to the best of our knowledge, none

of the previous smartwatch toolkits explored expanding the smartwatch with a diverse set of input and output functionalities.

2.5 Positionality Statement

Table 1 situates our work compared with other relevant works across smartwatch instrumentation, VLC and interactive capabilities imbued into smartwatches. It should be noted that typical approaches to enrich a smartwatch's interactive capabilities include adding a dedicated hardware stack [14, 28, 51]. However, when minimal instrumentation or in-built sensors are used [22, 47, 65], the interactive capabilities become limited to either input or output and communication often becomes unidirectional. LightLink offers flexible real-time sensing, low cost, simple circuitry, higher sensing range, two-way communication, and easy app integration. Our work expands smartwatch functionalities with minimal instrumentation. We are the first to implement visible light communication on a smartwatch using its built-in ambient light sensor (ALS) to enable communication with an external sensor add-on. Our system is ideal for on-demand sensing, real-time feedback, and eyes-free interaction.

3 WORKING PRINCIPLE

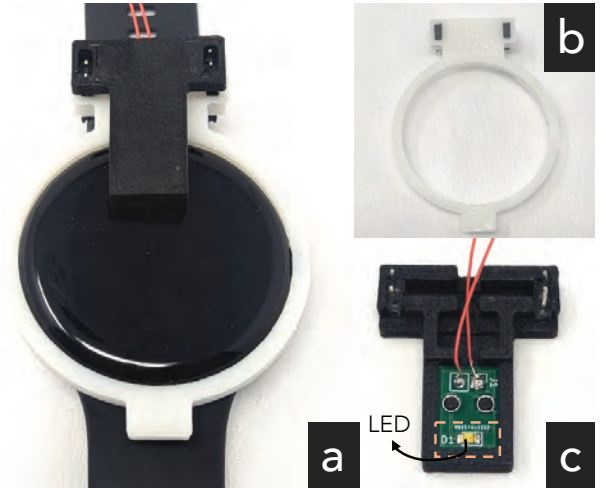


Figure 2: Visible Light Communication with Smartwatch through LightLink: (a) 3D-printed watch case mounted with an LED case focusing onto the ALS; (b) Watch case with female header pins to securely hold the LED case; (c) 3D-printed LED case with a PCB mounted with an SMD LED or a photodiode to get the data from a watch's screen.

Visible Light Communication (VLC) enables bidirectional data exchange using light as a medium. A VLC transmitter encodes information into modulated light signals—via On/Off keying, frequency or pulse modulation, or barcodes—using sources like LEDs or displays [24]. A receiver, such as a photodiode or a camera, captures and decodes the signals.

LightLink enables bidirectional Visible Light Communication (VLC) between a smartwatch and external add-on devices. Our system offers a low-cost, plug-and-play, lightweight setup with minimal instrumentation and without occupying much of the screen

space of the smartwatch (Figure 2a). For the **downlink**, the external device, equipped with a sensor (e.g., temperature or alcohol sensor), transmits data to the watch using an LED. This LED encodes information by modulating its brightness proportional to the sensor's readings. The smartwatch's ambient light sensor (ALS) captures the transmitted light intensity, which is then decoded to infer the sensed physical quantity and displayed on the watch. For the **uplink**, a small portion of the smartwatch screen acts as a light transmitter, while an external BPW34 photodiode serves as the receiver. The watch emits modulated light patterns that encode specific bitstreams. These are captured and decoded by a photodiode connected to a microcontroller, triggering actions on the external device. The watch interface can show different options to control the external add-on device.

To the best of our knowledge, this is the first work to explore how VLC can establish a communication link between smartwatches and external add-on devices, thereby expanding the interaction capabilities of wearable technologies.

4 CHARACTERIZATION EXPERIMENTS FOR LIGHTLINK

Smartwatches are equipped with a variety of onboard sensors (e.g., accelerometers, ambient light sensors), and their values are accessible through a software API. This API allows developers to register for sensor events and retrieve real-time sensor data. LightLink uses the Android API to register for and listen to light sensor events on a Google Pixel Watch 3, with the API reporting illumination levels as a single lux value. The primary function of the ambient light sensor is to monitor and automatically adjust the screen brightness for optimal viewing.

Before developing our communication prototypes, we conducted a series of technical experiments to characterize the ALS and watch screen performance as a light receiver and transmitter, respectively. These experiments were essential for designing and implementing the LightLink add-on devices and software pipelines.

The research questions for these experiments are:

- RQ1: Where is the Ambient Light Sensor in a Smartwatch located?
- RQ2: What is the optimal spectral frequency for an LED to communicate with the ALS?
- RQ3: What are the control strategies for efficient communication with the ALS?
- RQ4: What is the optimal colour of a watch's screen to transmit a bitstream?

4.1 Locating the Ambient Light Sensor

Old smartwatches and smartphones had dedicated light sensors on the bezel. In the latest devices, the ambient light sensor is positioned behind the display, making it invisible and reducing bezel size, enhancing aesthetics.

We conducted an experiment to detect the location of the ambient light sensor embedded behind a smartwatch screen. We used a Google Pixel Watch 3 for all our experiments, but these experiments can be generalized for other smartwatches as well. The light sensor

used in the Pixel Watch is a TPS3701 proximity and colour sensor chip.

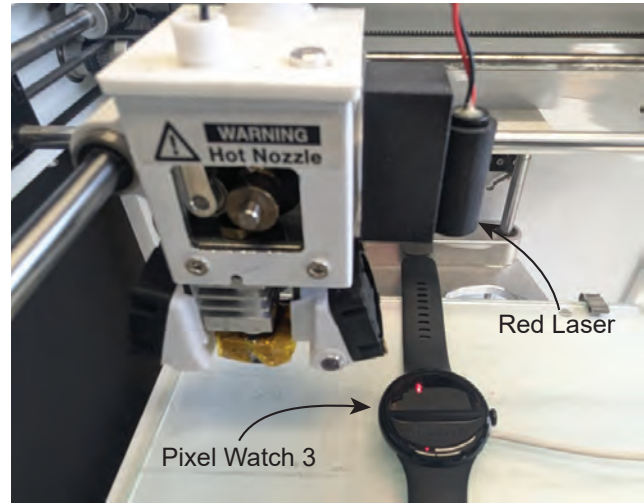


Figure 3: Apparatus for identifying the location of the Light Sensor. A laser attached to a 3D printer head was used to scan the upper half of the smartwatch screen to determine the location of the Ambient Light Sensor.

4.1.1 Method. The experiment setup consists of a 5mW 650nm red laser diode (1528-1391-ND) attached to the head of a 3D printer (Figure 3). A Pixel Watch 3 is placed on the build tray of a Ditto 3D printer about 8 cm away from the laser. Through initial trials, it was evident that the ALS is located in the upper half of the watch. So, the 3D printer head moves only on the upper half of the watch to precisely locate the light sensor embedded behind the screen. Figure 4 shows the black area (50 mm × 25 mm) highlighting the scan area. The 3D printer head moves with a stepsize of 1 mm, and at each point, the laser is turned on, the lux value from ALS is recorded and then turned off before moving to the next location. The experiment was performed in a dark environment to eliminate the influence of ambient light. A Python interface running on a host desktop sends G-code signals to control the movement of the printer's head and sends serial commands to control the laser. The watch sends the lux value to the host computer through WiFi. This setup allows the printer head to scan the smartwatch's surface and map the response of the ambient light sensor (ALS).

4.1.2 Results. Figure 4 shows a heatmap of the lux values measured across the upper half of the watch screen. The watch has a diameter of 45 mm, with the sensor located approximately 23 mm from the right edge and 9.5 mm from the top edge. We subsequently developed prototypes to shield this region of the watch from ambient light interference and create a dedicated channel to establish a communication link.

Takeaway: The ALS is located on the upper half of the smartwatch: 23 mm from the right edge and 9.5 mm from the top edge.

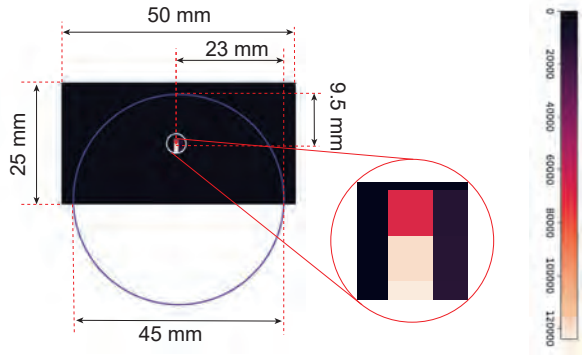


Figure 4: Heatmap of the smartwatch ambient light sensor on the Pixel Watch 3, with an inset showing a zoomed-in view of the sensor readings when the laser pointer directly hits the ALS

4.2 Identifying the Optimal Spectral Response

After locating the Ambient Light Sensor (ALS), we sought to identify the optimal LED color for communicating with it. Although the TCS3701 datasheet specifies its spectral response, those values were recorded under ideal conditions, not behind a smartwatch screen [1]. The TCS3701 captures light through six channels—Clear, Red, Green, Blue, Wideband, and IR—and computes a lux value using proprietary coefficients that vary based on different factors. However, the smartwatch API exposes only the final lux reading, not individual channel data or coefficient values. This experiment characterizes the ALS response to four LED colors to determine which produces the highest lux output with minimal current.

To communicate with the ALS, our experiment setup contains a 3D-printed case (Figure 2b) that goes around the periphery of the watch, providing a secure mount for an LED case that focuses light onto the sensor. The LED case (Figure 2c) houses a PCB that holds an 0805 SMD LED, along with two pairs of male header connectors for secure placement. The case is printed in black to prevent light from penetrating. The entire setup is positioned on the watch based on the results from the previous experiments, such that the LED shines directly onto the ALS sensors. We used the same setup (Figure 2a) for all the experiments in this section.

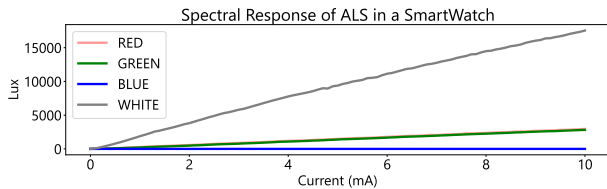


Figure 5: Spectral Response of the Ambient Light Sensor in a Smartwatch

4.2.1 Method. We experimented with four LED colors: Red, Green, Blue, and White to understand their correlation with light sensor values. The brightness of the LED is determined by the current flowing through it. An op-amp-based V-I converter controls the

LED current. A programmable MCP4822 DAC, interfaced with an Arduino UNO, generates voltages to set the desired current level for the V-I converter. The current is adjusted from 0 to 10 mA in steps of 0.1 mA, and the corresponding lux readings are logged. At each trial, the current is set to the desired level and then reset to zero before the next trial.

4.2.2 Results. Figure 5 shows that white LED has a higher slope, indicating greater responsiveness to small changes in current. This characteristic leverages the wide sensing range of the ALS, enabling higher resolution for sensing. So, we chose white LEDs to shine onto the ALS to facilitate communication from an external device to the watch.

Takeaway: White LEDs are ideal for transmitting data to the ALS because of their high responsiveness.

4.3 LED Control Strategy

During our initial exploration, we observed that the smartwatch does not trigger a light sensor event for subtle changes in LED current. So, we reset the LED current to zero before setting the desired current to eliminate this confound in the LED colour experiment. This experiment aims to understand in detail the correlation between LED current (white LED) and lux readings, which is important for interpreting data from the external sensor based on lux readings.

In the first condition, the current through the LED is linearly increased from 0 to 10 mA in steps of 0.1 mA without resetting to zero between two consecutive current settings. Figure 6 (ALS Linear green curve) shows that the lux value remains constant for subtle changes in LED current, resulting in a many-to-one mapping between current and lux. This non-linear behavior is undesirable for communication, as the watch cannot reliably detect small variations in LED brightness, likely due to the API triggering sensor events only when light intensity exceeds a threshold.

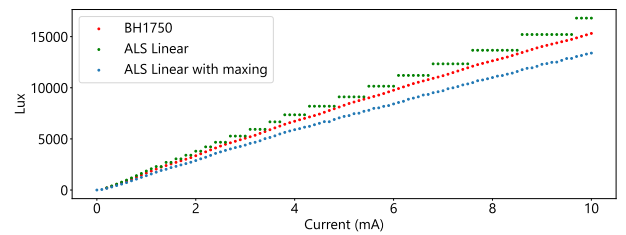


Figure 6: Lux vs Current Graph

To confirm that this behavior was due to the API thresholding and not typical of a lux sensor operation, we measured a current vs. lux curve using a standard BH1750 lux sensor. Figure 6 (BH1750 red curve) shows a standard lux sensor responds to small changes in current with the lux values linearly increasing with current. To overcome the API's thresholding limitation, the ALS must be reset between consecutive current settings. To reset, the LED current can be set to extreme values (0 mA or maximum current) before setting the desired LED current that communicates the external

sensor value. This forces a significant change in light intensity, continuously triggering the ALS. In the third condition, the LED current is reset to the maximum current (13.5 mA) before setting the desired current (0 to 10 mA). Figure 6 (ALS with maxing blue curve) shows the watch's ALS provides a linear increase in lux values for small changes in current. The circuit design of the sensor add-on should account for this to enable effective communication through the ALS.

Takeaway: The smartwatch ALS exhibits thresholding behavior due to its API, which can be overcome by resetting the LED to an extreme current value before transmitting data. This reset ensures consistent responsiveness and allows effective, linear communication of sensor values.

4.4 Optimal Screen Color for Bitstream Transmission

We intend to use the photodiode as a basic color sensor to detect two distinct flashing colors from the watch screen. To determine these two distinct flashing colors, this experiment characterizes the photodiode's response to different screen colors. While the manufacturer's datasheet provides its ideal spectral response, it is important to evaluate its response with the colors displayed by the watch screen.

4.4.1 Method. We used a simple square Textview (60dp × 60dp) in Android to transmit the light signals. The background of the Textview is changed to simulate different screen colors and the corresponding photodiode response is recorded. We experimented with six different screen colors (Black, White, Red, Green, Blue, Yellow) in the visible light spectrum. For each color, the amplified output of the photodiode is recorded. The experiment was conducted in a dark enclosure with the screen set to 100% brightness.

Color	Sensor Voltage (V)
BLACK	0.039
WHITE	3.363
RED	1.681
GREEN	1.901
BLUE	1.105
YELLOW	3.363

Table 2: Screen flashing color and sensor voltage

4.4.2 Results. Table 2 shows the BPW34 photodiode's response to different screen colors. The results show that black color results in the lowest voltage (0.039V), while white/yellow generates the highest voltage (3.363). To represent logic 1 and logic 0, we select the two colors with the largest voltage difference. Thus, black is used for logic '0' and white for logic '1'.

Takeaway: Black and white are used for logic 0 and 1 due to their maximum voltage contrast on the BPW34 photodiode.

5 PROTOTYPING TOOLKIT

To help readers replicate our design, we now provide the necessary technical details. To accelerate replication, we provide all the source codes, Android Library, and PCB design of our implementation¹. The key components of our LightLink are: (1) Watch Case: Houses the PCB for delivering power and communication to add-on devices, (2) Charging Case: This is used to charge the watch and the LightLink's watch case. (3) Software Library: To enable rapid and easy prototyping of LightLink applications, we contribute an Android Library that enables practitioners to set up the communication link and integrate LightLink add-ons into custom applications.

5.1 Watch Case

The smartwatch's built-in battery can't power external add-ons. To solve this, we developed a custom 3D-printed watch case (Figure 7a) that attaches to the smartwatch to power and communicate with add-on devices. The top section houses a switch PCB, 30 mAh battery, and female header connectors for mounting the add-ons. The switch PCB has a slide switch (CSS-1210TB) to enable/disable power to add-ons. The watch case includes charging terminals. The bottom half has a 4-pin connector for add-ons on the strap or forearm, with pins for power and signal control (Figure 7b). This design integrates external sensing or actuation modules seamlessly.

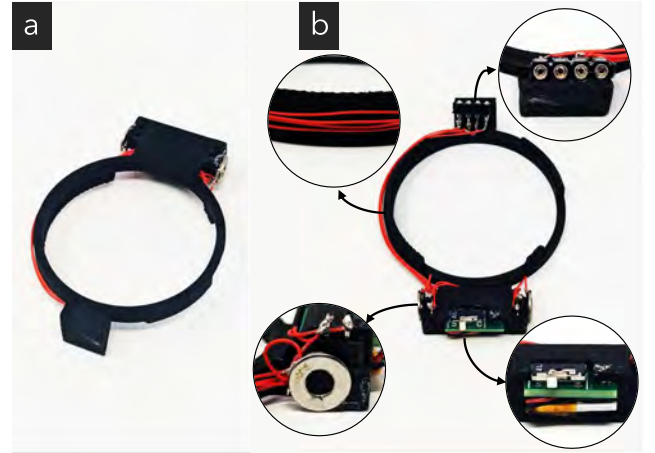


Figure 7: Watch Case: (a) a fully assembled 3D printed watch case for attaching add-ons; (b) Backside of the watch case showing a 4-pin connector to attach strap and forearm add-ons, charging contacts, battery PCB with a slide switch, and four wires routed around the watch case for interfacing the 4-pin connector to the top section.

5.2 Charging Case

We developed two variations of the charging system to recharge the watch case. The first design enables simultaneous charging of both the watch case and the smartwatch, allowing users to charge the smartwatch without removing the add-on device (Figure 8a). This setup consists of a 3D-printed charging pod that securely holds the existing smartwatch charging dock while incorporating

¹<https://github.com/antonyalbertraj1994/LightLinkToolkit.git>

two magnetic charging terminals for the watch case. An Adafruit MCP73831 LiPo battery charger is embedded in the base of the 3D-printed pod, with a small cutout to view the charge status LEDs.

To facilitate simultaneous charging, the smartwatch charging cable is modified to route the 5V and ground signals to a battery charger module. When the smartwatch is placed on the charging pod, the magnetic connectors align with the charging terminals on the watch case. The smartwatch itself charges as usual through its designated charging dock. Additionally, we designed a standalone charging system specifically for the watch case, allowing it to be charged separately when detached from the smartwatch (Figure 8b).

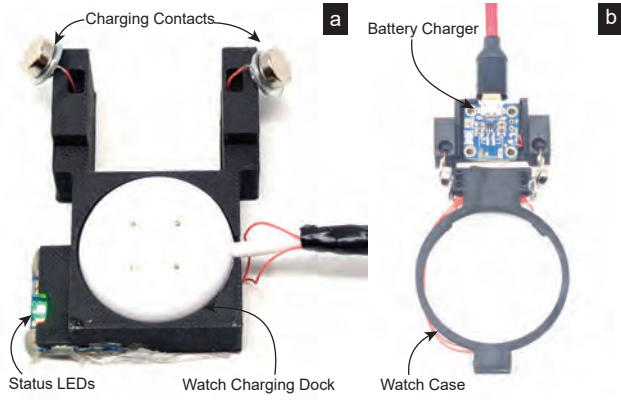


Figure 8: Charging Case: (a) Modified charging dock of pixel watch to charge both the watch and watch case simultaneously; (b) standalone charging case connected to a watch case through the magnetic contacts.

5.3 Hardware System

An LED driver circuitry is required to communicate information from an external sensor to the watch. We designed a suite of add-ons that consists of digital input sensors (e.g. 1-bit switches), analog sensors (e.g. alcohol sensors), visual output (LED array) and haptic output devices (vibrotactile actuator band). Below, we describe our approaches for modulating the LED brightness for digital and analog sensors.

5.3.1 Digital Transmission: A simple communication method between an external add-on and a smartwatch uses a single-bit transmission. The LED acts as a binary transmitter, switching between fully ON and OFF states. The ALS detects changes in lux values and uses a predefined threshold to determine the digital state of the add-on. Transmitting multiple bits requires more complex circuitry, such as using a microcontroller to flash the LED at different brightness levels, encoding multiple bits. We demonstrate these transmission schemes in Sections 6.1 and 6.2.

5.3.2 Analog Transmission: Analog transmission requires modulation of the LED brightness to transmit continuous values. This could be achieved in two ways:

5.3.2.1 Voltage to Current Converter. : The LED switches between a known reference current (I_{ref}) and sensor current (I_{sensor}),

mapped to the analog sensor value. A 2×1 analog mux switch (SN74LVC1G3157) connects its input to V_{DD} and the analog sensor output V_T . The select pin, controlled by a timer IC (LMC555), generates a 1.52 Hz square pulse train with a 50% duty cycle, periodically switching the mux output between V_{DD} and V_T . The mux output connects to an opamp-based V-I converter (LMV321) with the LED as the load. This maps the input voltage to a proportional LED current. The current alternates between I_{ref} and I_{sensor} based on V_{DD} and V_T inputs. The smartwatch app decodes brightness patterns to interpret the analog value. This communication scheme is demonstrated in Section 6.3.

5.3.2.2 PWM Modulation: In our microcontroller-based system, LED brightness is modulated by adjusting a 10 kHz PWM duty cycle in proportion to the analog sensor value measured by the ADC. For the watch to predict the analog sensor value, a one-time calibration is required to establish the relationship between the ADC voltage and lux values. For this calibration, a DAC (MCP4822) generates voltages from 0 to 3.3 V, which are mapped to duty cycles between 0% to 100%, and the resulting lux values are recorded. Figure 9 (Initial Calibration Curve) shows that the relationship between ADC voltage and lux is non-linear at extremes (<0.1 V, >3.2 V). To address this, the ADC value is scaled between 5 to 95% duty cycle to overcome the non-linear effect. Figure 9 (Recalibrated Curve) shows a linear relationship between the ADC voltage range (0 to 3.3 V) and the lux value. Similar to the V-I converter scheme where the current alternates between I_{ref} and I_{sensor} , the microcontroller alternates the LED between 95% duty cycle (corresponds to ADC voltage at $V_{ref} = 3.3$ V) and a duty cycle mapped to the sensor value (V_{sensor}). This communication scheme is demonstrated in Section 6.4

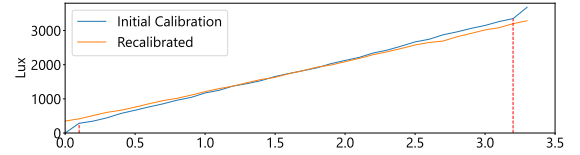


Figure 9: Calibration to characterize the relationship between ADC voltage and lux values

5.4 Software Calibration

At startup, the smartwatch app performs a 5-second calibration to determine the reference lux value ($Lux_{reference}$) corresponding to a known current (I_{ref} for V-I converter) or (V_{ref} for PWM approach). This is computed by averaging the local maxima of the lux readings observed during the interval. A scaling factor is then derived by comparing $Lux_{reference}$ to the lux value from the initial calibration curve at I_{ref} or V_{ref} , compensating for positional shifts.

An inverse first-order calibration curve ($I_{sensor} = f(lux)$) for V-I converter or ($V_{sensor} = f(lux)$) is fitted to estimate the current or ADC voltage from the lux values respectively. During operation, the system checks if it receives lux values within 2% of $Lux_{reference}$. If this fails for 10 consecutive events, the 5-second calibration routine is triggered again.

The system detects minima (Lux_{sensor}) between peaks ($Lux_{\text{reference}}$), then computes I_{sensor} or V_{sensor} from the inverse curve to estimate the current and voltage, respectively. For the V-I converter, the calculated current is used to determine the sensor voltage using $V = R_{\text{sense}} \cdot I$, where R_{sense} is the feedback resistance used in the V-I converter. This voltage is used to estimate the temperature (Section 6.3) using the sensor's datasheet transfer function. In the PWM-based approach, the sensor voltage value is directly predicted by the watch using the inverse curve.

5.5 Software Toolkit

We designed a software toolkit to help designers, hobbyists, DIYers, and HCI researchers create custom add-ons for watches. It includes an Arduino library to map sensor values to LED signals and Android helper functions to decode sensor values using the Ambient Light Sensor (ALS).

To connect an analog sensor (e.g. FSR) to a smartwatch, the user needs to follow the below procedure:

- Connect the sensor (e.g. FSR) using the traditional voltage divider circuit, and the output voltage needs to be read through the ADC of the microcontroller.
- Now, the ADC value should be mapped to LED brightness variations to communicate the signal to the smartwatch. To control the LED brightness, the LED current needs to be modulated, which can be done through (1) a resistor or potentiometer connected in series to the LED, (2) a programmable constant current driver, or (3) generating different PWMs to control the LED current. We use the latter in our case.
- Use our Arduino library function to send the appropriate signal control to the LED. The library is initialized as

```
LightLinkTransmit send(int analogPin, int ledPin,
    int commsSpeed, int maxSensorValue)
```

, where *analogPin* is the analog input pin on the microcontroller connected to the analog sensor, *ledPin* is the pin connected to the LED for sending modulated light signals, *commsSpeed* is the desired communication speed (which can be selected based on experiments in Section 7.2), and *maxSensorValue* is the maximum sensor reading (e.g., 1023 for an FSR connected to the 10-bit ADC on an Arduino Uno). Then calling the function

```
send.transmitAnalogData()
```

starts mapping the analog sensor value to modulated LED signals. This library is designed to simplify the process for novice developers, while experts can still create their own custom functions to generate these LED signals.

- Once the data is transmitted through the LED, our Android helper library can be integrated into any custom app to get the data from the Ambient Light Sensor. The *AnalogSensor* class (for V-I converter) or *AnalogSensor_Microcontroller* class (for PWM-based) overloads the light sensor event function callback and decodes the Lux values to interpret the analog sensor values.

Similarly, to send data from a watch to an external device, the user needs to follow these steps:

- The android library class *TransmitData* is initialized as

```
transmitData = TransmitData(activity:Activity,
    Brightness:Int )
```

where *activity* takes the reference of the Activity containing a Textview placed appropriately underneath the photodiode, and *Brightness* is the desired screen brightness (Refer to Section 7.1 for appropriate values).

- Then, calling the function

```
TransmitData.startSendingArray(dataString:String,
    textView:View)
```

will start sending the flashing signals, where *dataString* is the 8-bit data string to be sent, *textView* reference of the TextView, and *bitLength* representing the length of each flashing pulse (refer to Section 7.2 for these values).

- After setting up the Android Application, the Arduino library will help decode the flashing signals received by the photodiode. The Arduino Library is initialized as

```
BitReader reader(int analogPin, int threshold, int
    bitLength)
```

where *analogPin* is the pin for connecting the amplified output from the BPW34, *threshold* to detect logic '0' and '1' (usually set to half of max ADC value 1023/2), *bitLength* is the length of flashing pulse sent from the watch. Then calling the function

```
reader.readBits()
```

returns a pointer to an array of 8 integers, with each value corresponding to a bit sent from the watch.

6 APPLICATION-DRIVEN VALIDATION

Toolkits are notoriously difficult to evaluate in HCI [15, 33, 44]. The predominant method of evaluation, the controlled usability study, is a poor match for toolkits, as usability studies tend to focus on a narrow subset of a toolkit's capabilities [33, 44], rarely aligning with "how [the system] would be adopted and used in everyday practice" [15]. As recommended by Ledo et al., we adopt application-driven validation [33] to evaluate the versatility of our toolkit in supporting diverse add-on interfaces.

Our validation demonstrates the following capabilities of LightLink:

- Supporting diverse add-ons: LightLink enables the augmentation of digital sensors, analog sensors, infrared (IR) transmission, and visual and haptic output.
- Supporting Diverse Locations: We demonstrate that these add-ons can be augmented to smartwatches at watch strap, bezel, and on the body.
- Low-Cost: All our add-ons are low-cost, with the most expensive add-on costing ~\$11 USD.

Table 3 summarizes the application demonstrators we implemented to augment smartwatches with diverse capabilities cost-effectively.

6.1 Tilt sensor

We present a simple application of a one-bit device using a passive mercury tilt switch to count repetitions in various gym exercises. For example, a tilt switch attached to the forearm (Figure 1c) can track the number of dumbbell rows or bicep curls in a set. As the user moves their arm, the tilt switch alternates between open and

Add-on	Type	Transmission	Location	Cost	Power	Runtime	Application
Tilt Sensor	Sensor	1 bit	Forearm	\$0.51	7.6 mW	~15 hours	Fitness Tracking
Buttons	Sensor	4 bits	Watch Strap	\$4	23.1 mW	~7 hours	User Interface
Temperature	Sensor	Analog	Watch Strap	\$3.5	18.5 mW	~6 hours	Health Monitoring
Alcohol Sensor	Sensor	Analog	Bezel	\$6.6	115.5 mW	~4 hours	Health Monitoring
Color Sensor	Sensor	Analog	Bezel	\$6.7	23.95 mW	~5 hours	Health Monitoring
IR Remote Control	Data Transmission	8 bits	Bezel	\$4.1	15.84 mW	~7 hours	Device Control
4x4 LED Matrix	Visual Output	16 bits	Watch Strap	\$11.5	184 mW	~3.5 hours	Notifications
Vibration Band	Haptic Output	4 bits	Forearm Band	\$8.5	475 mW	~4.3 hours	Haptic Feedback

Table 3: Application Demonstrators to validate LightLink’s capability to cost-effectively support a diverse set of add-ons. Note, the reported power consumption accounts for both the communication circuitry and the sensor or actuator.

closed states, causing the attached LED to switch between high and low brightness levels. The ambient light sensor (ALS) detects these brightness changes, which are processed to display the repetition count in real time. For this use case, we use a simple thresholding scheme on the Android application side to detect the ON and OFF states of the LED. Vibrotactile feedback is triggered through the smartwatch for each repetition to add expressivity. At the nominal voltage of 3.7 V from a LiPo battery, the power consumption of this add-on is 7.622 mW.

6.2 Physical Buttons for Eyes-Free Interaction on Watch Strap

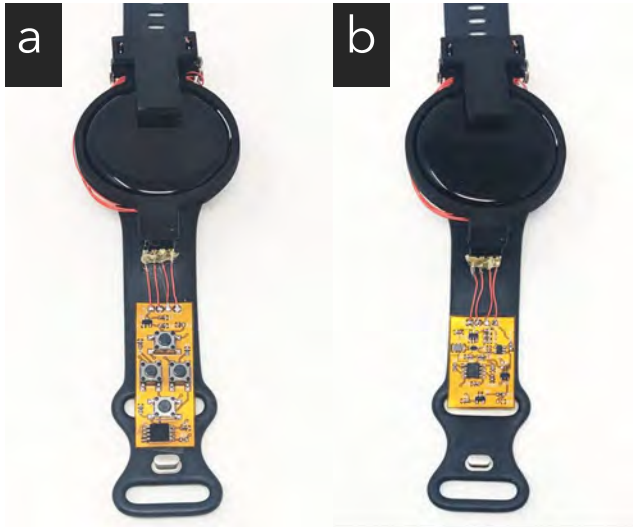


Figure 10: Buttons and Temperature Strap Add-on: (a) Four buttons mounted on a flexible PCB and attached to the strap, and connected to the watch case for power and communication with the LED; (b) flexible temperature sensor add-on PCB attached to the watch strap for on-demand body temperature sensing

To demonstrate multi-bit digital transmission, we designed a flexible PCB (17.5 mm × 47 mm) with four tactile buttons for attachment to the watch strap (Figure 10a). The flex PCB is powered via the watch case header pins, with a MIC5504 3.3V voltage regulator

providing a stable supply. An ATtiny85 microcontroller interfaces with the buttons, detecting their states and generating unique PWM pulses to vary LED brightness based on the button pressed. The smartwatch decodes these brightness variations to identify the button. The circuit consumes 15.279 mW in standby mode and up to 23.1 mW when a button is pressed. The bill of materials costs around 4 dollars. This add-on enables eyes-free interaction for tasks like controlling music when the watch face is hidden or changing music tracks while biking.

6.3 Temperature Sensor

Some smartwatches monitor skin temperature but cannot accurately reflect core body temperature due to the influence of external factors. We designed a sensor that measures core body temperature by placing the watch strap on the forehead.

6.3.1 Hardware Implementation: The temperature sensor circuit is built on a 31 mm × 21 mm flexible PCB that mounts onto the watch strap (Figure 10b). When connected to the watch case, it receives power, which is boosted to 5 volts using a TPS613222ADBVR boost converter, serving as the system-wide voltage. We used a TMP236 analog temperature sensor that measures temperatures from 0°C to 70°C, with an accuracy of ±0.5°C accuracy. The sensor outputs a voltage signal proportional to the temperature with a sensitivity of 19.5 mV/°C. We used the V-I Converter scheme mentioned in Section 5.3 to implement the analog transmission scheme. The bill of materials costs \$3.5 and consumes 18.5 mW at room temperature, with power consumption varying with temperature.

6.3.2 Validation: To validate our prototype, we conducted an experiment comparing the temperature measurements from our sensor add-on with a baseline reference. The baseline temperature was obtained using the same TMP236 sensor interfaced with an Arduino Uno, while different temperature conditions were simulated using a hotbed. Figure 11 shows that our sensor add-on can accurately track temperature variations.

6.4 Alcohol Sensor

Commercial smartwatches and smartphones lack integrated alcohol sensors. We designed a wearable alcohol sensor add-on (Figure 12a) that measures breath alcohol content on-demand, providing a portable solution without needing a separate breathalyzer.

The alcohol sensor add-on consists of a PCB measuring 18.5 × 13.5 mm mounted onto the LED holder case (Figure 12a). We used

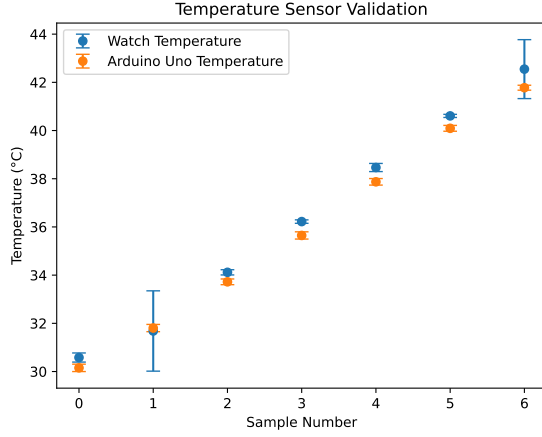


Figure 11: Temperature Sensor Validation

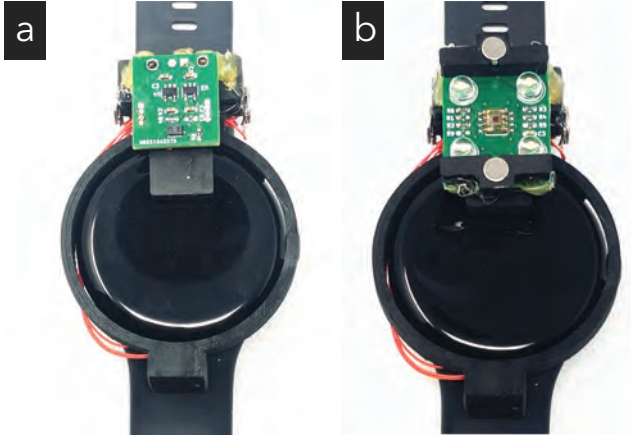


Figure 12: Alcohol and Color Sensors Bezel Add-ons: (a) Alcohol sensor add-on with a CCS 803 ethanol sensor mounted on a PCB, and attached to the LED holder for on-demand breath alcohol sensing; (b) Color sensor add-on with PCB mounted with a TCS 3200 color sensor and 4 LEDs to illuminate a sample, and attached to a LED holder.

a CCS803 metal oxide alcohol sensor, which generates a voltage proportional to the level of detected alcohol. The CCS803 has a heating element that must be pre-heated before it can sense alcohol levels. A 1.8V voltage regulator powers the heating element. The CCS803 sensor's resistance changes in proportion to the alcohol level detected. This change is incorporated into a voltage divider, with the output connected to the ADC of the ATtiny 85 microcontroller. The microcontroller is powered by a separate 3.3V voltage regulator (MIC5504 3.3V). The microcontroller generates a PWM signal to encode the alcohol sensor value into LED brightness. We used PWM modulation (described in Section 5.3) to transmit the analog values. The bill of materials costs 6.6 dollars and power consumption is 115.5 mW. Assuming a 20-second heating period and a 10-second measurement period, the device can last for 102 detection cycles on a single full charge of the watch case battery.

6.4.1 Validation. We conducted a validation experiment to evaluate the ability of the watch to predict the input voltage to the ADC. The input to the ADC can be from an analog sensor like the Alcohol sensor or any analog sensor. To simulate different voltages from an analog sensor, the DAC generated voltage from 0 to 3.3V in 100 mV increments, and the corresponding watch's predicted voltage was recorded over 10 trials.

Figure 13 shows the comparison between the set voltages and the watch's predicted values. The watch is able to accurately predict voltages in the range of 0 to 2.9 V. Beyond 2.9 V, the LED brightness variation is too subtle to trigger the ALS. To overcome this, the analog sensors can be calibrated to generate output within the 0–2.9 V range to ensure accurate voltage prediction by the watch.

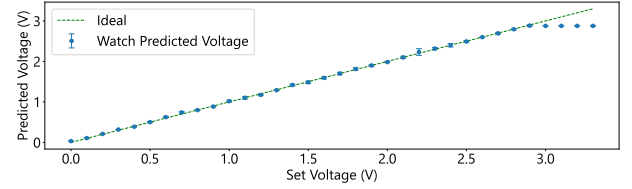


Figure 13: Set Analog Voltage vs Watch's Predicted Analog Voltage

6.5 Color Sensor

Color sensing on wearable devices can assist diabetic patients with glucose strips but often requires dedicated devices or manual color matching, which is inconvenient and challenging for those with color blindness. In some systems, continuous monitoring is needed for accurate quantification [49]. The TCS3701 ambient light sensor in smartwatches has color sensing channels, but the smartwatch API doesn't provide direct access, limiting its use for color-based applications.

6.5.1 Hardware Implementation. The color-sensing add-on consists of a rigid PCB attached to the 3D-printed LED holder (Figure 12b). The PCB measures 28 mm × 21.5 mm and contains a TCS3200 color sensor, an Attiny 85 microcontroller, and a voltage regulator (MIC 5504 3.3V) to power the onboard circuitry. The add-on attaches to the watch bezel for power. Samples are placed inside the removable top lid for sensing. The smartwatch app starts, and the watch case switch is turned on to sense and communicate the color information.

The TCS3200 color sensor detects color through four channels: Red, Green, Blue, and Clear. Each channel produces a frequency output proportional to the intensity of reflected light, which is measured by a microcontroller. The microcontroller controls an LED to transmit color data to the smartwatch using PWM modulation (described in Section 5.3). The LED communication sequence begins with a 90% duty cycle for 300 ms, followed by a 0% duty cycle for 300 ms, repeated three times for calibration. After calibration pulse, the duty cycle is adjusted based on the detected frequencies for the red, green, and blue channels. Each channel's data is flashed for 300 ms, with a 90% duty cycle delimiter between channels.

This structured sequence ensures that three distinct values are transmitted to the smartwatch, which decodes the brightness pattern. When the watch detects a start pulse with a high-brightness signal from the LED, the lux values are recorded at 10-second intervals. The watch then computes the value for each channel as the ratio between the lux value corresponding to a channel and the reference lux (Lux_{ref}). These values are matched with a lookup table to identify the closest color match. An initial calibration step is required to generate this lookup table for accurate color identification. The system has a bill of materials costing 6.7 dollars, consumes an average power of 23.95 mW of power for a 10-second color detection cycle, and can perform 1440 detection cycles on a fully charged watch case battery.

6.5.2 Validation. To validate the color sensor add-on device, we conducted a color analysis using 4 simulated color strips printed on standard A4 paper. The color used in the experiment are yellow, green, blue, and red. An initial calibration was performed to obtain ground truth values from the color sensor, which were used to generate a lookup table for each color. Each color sample was tested 5 times, and the color were detected successfully in all the trial, except blue failing in one of the trials. The TCS3200 is obsolete, so we extracted the chip from an evaluation board. As an alternative to using a dedicated color sensor like the TCS3200, a simple LED setup combined with arrays of Light Dependent Resistors (LDRs) [49] can also be used to detect color changes. This approach can be easily integrated into our system.

6.6 IR Remote Control

Older smartphones were equipped with an infrared (IR) transmitter LED for wirelessly transferring files or controlling nearby IR-enabled devices, such as televisions and air conditioners. However, smartwatches lack an IR transmitter due to size constraints. So, we designed an IR remote control add-on that attaches directly to the smartwatch, enabling convenient control of nearby IR-enabled appliances.

The IR remote add-on contains a rigid PCB (18.5 mm × 13.5mm) that houses the communication circuitry (Attiny 85, BPW34, and LMV 321), and a 940 nm SMD IR LED (IR12-21C/TR8) to send IR codes. The BPW34 photodiode is placed on the bottom of the PCB to receive signals from the smartwatch's screen. This assembly is enclosed in a 3D-printed case with male header connectors to attach to the watch case and receive power (Figure 14a).

The smartwatch app (Figure 14b) displays a list of devices that can be controlled (e.g., TV, Camera, Robot). Upon selecting a device, the app presents the relevant parameters that can be adjusted, such as increasing or decreasing volume. Based on the selected device, the app sends flashing light signals, which are picked by the BPW34 photodiode and decoded by communication circuitry. The microcontroller controls the IR LED to transmit the IR commands received from the watch, using the standard 38 kHz SIRC protocol. These modulated IR signals can be detected by a standard IR receiver (TSOP382) and decoded by a microcontroller to execute various actions. The IR remote add-on can also directly control an IR-enabled device (TV, Fan) if appropriate IR codes are programmed into the watch app.

The standby power consumption of the system is 14.6 mW, and consumes 15.84 mW when transmitting an IR command for a brief moment. The bill of materials for the device costs 4.1 dollars. With our watch case, the device can run for approximately 7 hours on standby mode.

6.7 Strap LED Display

Viewing notifications on the smartwatch screen can be inconvenient while walking or biking, as it often requires flicking the wrist to see the screen. Additionally, it becomes more inconvenient when the watch is hidden behind clothing, requiring users to adjust the sleeve to access the screen. An additional display on the watch strap can provide a convenient viewing angle for notifications and navigation directions while on the go. We designed an LED display add-on for the watch strap that shows basic notifications and animations. The display can shine through fabric, eliminating the need to adjust the sleeve to view information.

The LED display add-on contains 16 LEDs (LSM0805543V) arranged in a 4 × 4 LED grid on a flexible PCB (54 × 19.5 mm). The PCB includes communication circuitry (same as IR remote) and a constant current LED driver (STP16CP05) to drive the 16 LEDs (Figure 14c). The current flowing through the LED is set using a programmable resistor on the LED driver. The watch screen flashes a 16-bit serial signal, which is then decoded by the communication circuitry to activate or deactivate the appropriate LEDs. Figure 14d shows the LED display add-on mounted on the watch strap and connected to the watch case. The display can show animations to indicate navigation directions and render basic characters to represent notifications from different applications. With more engineering, the device can be extended to a denser LED array to show more detailed information.

The system has a standby power consumption of 24.75 mW (All LEDs off). It consumes 103 mW to power 8 LEDs and 184 mW to power all 16 LEDs. In a typical real-world scenario, the display turns on briefly to show the notifications. For instance, if all the pixels are turned on for 5 seconds every 5 minutes to display notifications, the device can operate for approximately 3.5 hours on a single charge. The total bill of materials for the device costs 11.5 dollars.

6.8 Vibration cues around the wrist

Smartwatches typically use a single vibration motor to generate vibration alerts on the wrist. This haptic feedback is limited to the wrist area underneath the watch. However, additional vibration motors can be placed around the forearm to provide richer feedback. We designed a vibration band for the forearm that can be controlled from the watch to play different patterns (Figure 14e). It can be used in eyes-free navigation while walking or biking, provide alternative sensory cues for individuals with disabilities, or offer calming patterns for meditation.

The vibration band consists of four vibration motors (FIT0774) attached to an elastic white band. Each motor measures 9mm in diameter and 2.7mm in height. These motors are connected to a custom 93 × 18mm flex PCB, which includes a communication circuit, four transistors (MMBT2222), and four flywheel diodes (1N4148). The microcontroller controls the motors by switching the transistors on or off. The smartwatch sends control signals

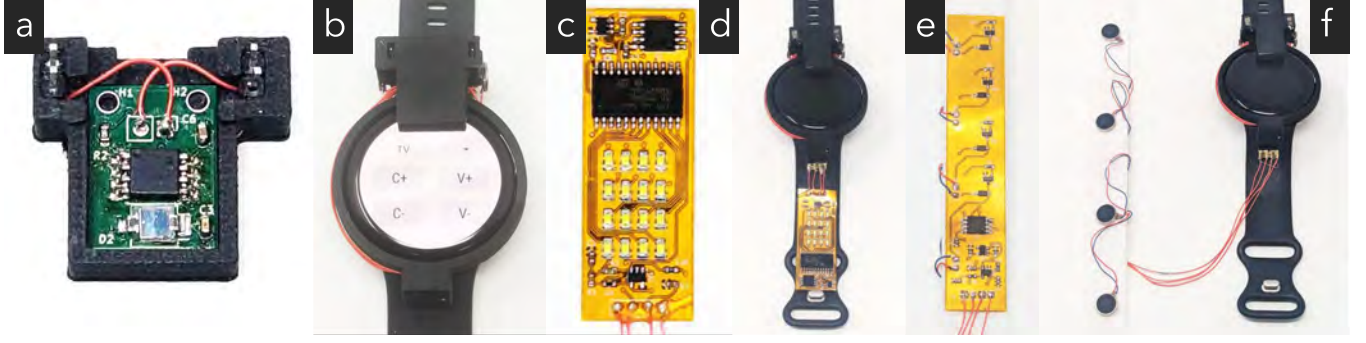


Figure 14: Output Hardware Prototypes: (a) Custom PCB housing a IR remote control add-on; (b) IR transmitter control interface on the smartwatch; (c) 4×4 LED matrix mounted on a flex PCB with microcontroller and LED driver; (d) LED add-on module mounted on the watch strap and connected to the watch case; (e) Flex PCB mounted with a microcontroller to control the vibration motors; (f) Vibrations motors mounted on a flexible cloth band and connected to watch case.

by flashing the display, which are decoded by the communication circuitry to activate the appropriate motors. Figure 14f shows the haptic band on-connected to the watch case.

The system's standby power consumption is 14.85 mW (all motors off), 265 mW to drive two motors, and 475 mW to drive four motors. If all four motors are activated for 5 seconds every 5 minutes, the system can operate for approximately 4.3 hours on a single charge. The bill of materials costs 8.5 dollars.

7 TECHNICAL EVALUATION

To characterize the performance of LightLink we conducted technical experiments. Our main research questions were:

- How does screen brightness impact data communication?
- What is the reliable communication speed of LightLink?
- How does ambient lighting influence data communication?

7.1 Influence of Screen Brightness

Since the ALS is located beneath the watch screen, its lux readings are affected by factors such as the distance to the cover glass, its thickness, refractive index, and screen brightness. While these variations are tolerable for automatic brightness control, they significantly impact our application, which relies on accurate ALS measurements. Screen brightness also influences LightLink's photodiode's (BPW34) response, but running the display at maximum brightness is power-inefficient. Thus, it's essential to identify an optimal brightness level that ensures a strong voltage contrast between black and white flashes—sufficient for reliable demodulation—while minimizing battery drain.

7.1.1 Method. We measure the ALS readings at three screen brightness levels: 0%, 50%, and 100%. For each level, the LED current vs. Lux curve was computed. The 0% setting does not completely turn off the display but maintains it at minimum brightness. To evaluate the response of LightLink's photodiode (BPW34), we experimented with 10 different brightness levels from 0 to 100% in steps of 10. For each brightness level, the output amplified voltage of the BPW34 is recorded. The experiment is conducted for both black and white flashing colors.

7.1.2 Results. Figure 15 illustrates the relationship between lux and LED current at different screen brightness levels. We excluded lux readings below 100, treating them as noise. A first-order curve fit was applied, and the root mean square error is shown on the graph. The results indicate that the ALS provides more reliable readings at lower screen brightness due to reduced fitting error. The predicted sensor readings remain visible on the watch screen even at the 0% brightness. At 50% and 100% brightness, the readings fluctuate more, especially at higher LED currents. For the BPW34 photodiode (Figure 16), we observed that the voltage difference between black and white flashes saturates at 40% screen brightness, eliminating the need to operate at full brightness. This makes 40% brightness an optimal setting for decoding modulated light signals reliably.

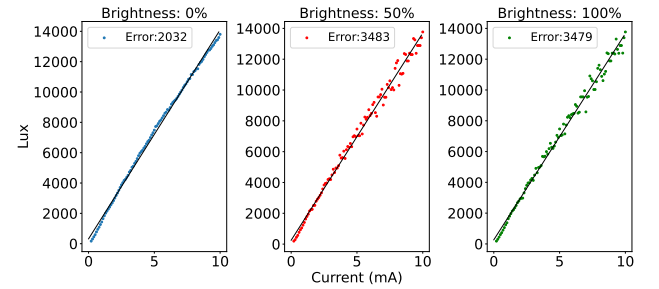


Figure 15: LED Current vs Lux for different screen brightness

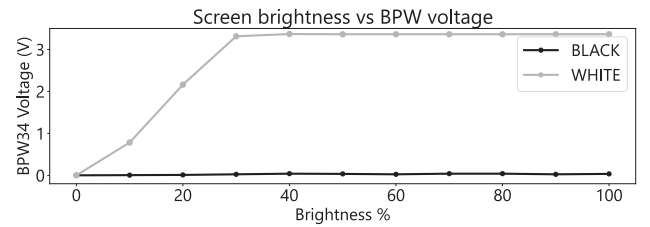


Figure 16: Screen Brightness vs PhotoDiode Voltage Value

7.2 Communication Speed

The Android API streams lux values using event-driven methods, which prevents fixed-rate sampling. Additionally, ambient light sensors typically require an integration time to compute light intensity [1]. These two factors limit the maximum achievable communication speed. However, it remains unclear how quickly the smartwatch's ambient light sensor (ALS) responds and how reliably it detects incoming light pulses. This responsiveness ultimately defines the communication speed between an external add-on sensor and the watch.

For the uplink (via the BPW34 photodiode), we also evaluated the transmission speeds for sending a single 8-bit packet. Like serial communication protocols used in microcontrollers, the system transmits 8-bit data packets with start and stop bits, totaling 10 bits.

To generate flashing signals, we used the Choreographer method², which synchronizes updates to the TextView background with the smartwatch's 60 Hz display refresh rate. The Choreographer triggers a callback every 16.67 ms (i.e., once per frame). By adjusting the number of callback cycles between updates, we control the bit length—for instance, updating the display every two cycles results in a bit length of 33.3 ms (2×16.67 ms). The shortest bit length the photodiode can reliably decode determines the maximum data rate. Consequently, transmitting a single 8-bit packet (1 start bit + 8 data bits + 1 stop bit) takes 10 times the bit length.

7.2.1 Method. We conducted an experiment to determine the maximum triggering rate at which the watch's light sensor can accurately detect and decode light pulses. The LED is flashed at different frequencies ranging from 3 Hz to 20 Hz, sending 100 on-off pulse cycles at each frequency. The LED is turned on and turned off completely in each cycle. The lux values reported by the watch are recorded during this sequence.

We measured data transmission with BPW34 by experimenting with six different bit lengths: 16.67 ms, 33.3 ms, 50 ms, 67.0 ms, 89.0 ms, and 100 ms. The time to transmit one 10-bit serial packet (1 start bit, 8 data bits, and 1 stop bit) for these bit lengths is 166.7 ms, 333 ms, 500 ms, 570 ms, 890 ms, and 1000 ms, respectively. We programmed the ATtiny microcontroller to transmit a fixed 8-bit data packet (0xAA, 10101010) due to its alternating pattern of '1' and '0'. This choice was made because the ATtiny lacks a serial port, preventing dynamic verification of successful data reception and decoding. After receiving the 0xAA packet, the ATtiny sets a digital pin high to signal successful transmission, and the Arduino Uno tracks the number of successfully received packets.

For each speed condition, the smartwatch transmitted 50 packets of data. The transmission success rate was then calculated based on the number of correctly received packets out of the 50 sent. The watch screen is set to 100% brightness with no external ambient light falling on it.

7.2.2 Results. The lux readings are decoded by counting the times the lux values are high and low. The ideal value is 200 because there are 100 ON-pulses and 100 OFF-pulses. The threshold was fixed because the LED was either entirely ON or OFF, and we were not concerned with the pulse duration. Instead, we focused on the sensor's ability to detect transitions between high and low states.

²<https://developer.android.com/ndk/reference/group/choreographer>

The results, shown in Figure 17, indicate that the watch's light sensor can reliably detect state changes from 3.33 Hz to 10 Hz, with a maximum triggering speed of 10 Hz. This value is critical for circuit design as it defines the maximum rate at which the ALs can be triggered to communicate external sensor values to the watch.

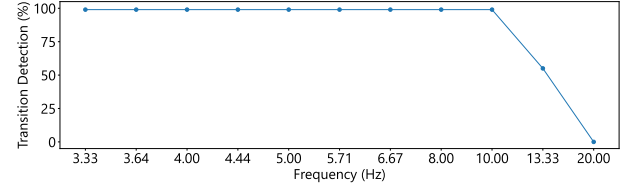


Figure 17: Transition rate for ALS sensor: Frequency vs Success Rate

Figure 18 shows the correlation between transmission speed and successful transmission rate. The results show that the highest achievable transmission 167 ms per packet (equivalent to 6 Hz) yielded an 88% percent success rate. For all the other transmission speeds, a success rate of 98% was achieved, with only the first packet failing to transmit successfully. This result shows there is only 2% of the packet lost for transmission speeds ≥ 333 ms per packet.

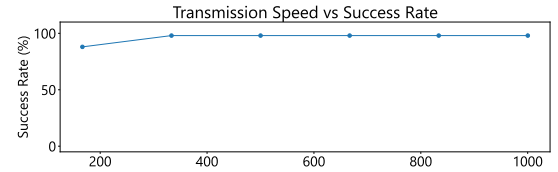


Figure 18: success rate vs Transmission Speed per packet (ms)

7.3 Effect of ambient Light

To ensure reliable communication using LightLink, it is essential to minimize the impact of ambient light on the ALS or photodiode readings. We conducted this experiment to assess whether our 3D-printed LED/photodiode holder effectively isolates the watch's ALS and the external photodiode from ambient lighting, thereby ensuring reliable communication.

7.3.1 Method. We used a Philips Hue white light with adjustable brightness to simulate various ambient lighting conditions. A BH1750 lux sensor was placed next to the watch to measure the lighting levels. The experiment was conducted across five different lighting levels, representing real-world usage scenarios. To evaluate the stability of ALS sensing, we varied the LED current from 0 to 10 mA in 0.1 mA increments and recorded the corresponding lux values. We then calculated the average change in sensor readings relative to a dark room (no ambient light). For assessing communication reliability with the BPW34, we set the screen brightness to 100% and the data transmission speed to 333 ms—the fastest speed with high success rate of transmission determined from the previous experiment, and measured the success rate of sending data from the watch to the external add-on. To reduce external light interference, the experiment was conducted inside a closed enclosure.

Light Setting	Avg. Lux	Avg. Error (ALS)	Success Rate (BPW34)
Dark Room	1	0	98
Living Room	146	0.6	98
Office Space	499	0.6	100
Overcast Day -1	989	0.5	100
Overcast Day -2	2905	0.5	100

Table 4: Effect of Ambient Light on LightLink transmission

7.3.2 Results. Table 4 shows the average lux values measured by the BH1750, the average error of ALS readings relative to a completely dark condition, and the transmission success rate under different lighting conditions. The ALS readings show an average error of less than 1 with respect to the dark conditions, and the photodiode reliably decodes signals from the watch with a success rate of 98% or higher across all conditions. These results confirm that the 3D-printed holder effectively shields the ALS and photodiode from ambient light, enabling reliable communication between the watch and external add-ons.

8 DISCUSSION, LIMITATION & FUTURE WORK

This section discusses how our approach can be extended to other smartwatches, as well as to devices beyond smartwatches, and addresses the limitations and potential directions for future work.

Generalizing to other Smartwatches — Although all experiments were conducted on a single device (Google Pixel Watch 3), our characterization focused on the ambient light sensor (TCS3701), which is also used in other smartwatches, including the Apple Watch Series. The Samsung Galaxy Watch series uses a similar sensor from the same manufacturer (AMS OSRAM), specifically the TCS3400. While the sensor model and placement may vary across manufacturers, our characterization setup can be used to detect the ALS location and perform calibration on other smartwatch models. For instance, testing with a Samsung Galaxy Watch 3 showed the ALS positioned at the bottom edge of the screen, which remains compatible with LightLink. Future work can also explore how the position of ALS, spectral response and ambient light affect the performance of LightLink. Based on this, a model could be developed to automate the entire calibration process.

Generalizing Beyond Smartwatches — Ambient light sensors are found in many modern touchscreen devices. This technique can be expanded to augment add-on interfaces for commercial touchscreen devices. For example, an air quality sensor could be integrated into a car dashboard display to communicate the air quality inside the vehicle. It could also be extended to communicate with phones and tablets, enabling the addition of external devices.

Better Hardware setup — The current LightLink setup requires a separate watch case and an LED/photodiode holder for communication with the watch. Currently, the LED holder must be removed to access the full watch screen. A more sophisticated mechanism could allow the holder to slide into the watch case from the top during regular use and be reattached when needed. Additionally, the wires connecting the strap sensors to the top of the watch run along the exterior of the case. Future versions could embed these wires

within the case using 3D-printed traces with conductive filaments or fabricated traces with conductive ink, resulting in a sleeker and more user-friendly design.

Further explore the built-in ALS — Similar to ViBand [32], developing a custom kernel to access the hardware registers of the ambient light sensor (ALS) could help enhance and stabilize its sampling rate or modify its behavior. This could open up possibilities for exploring different modulation schemes for communication with the ALS. Furthermore, gaining access to the individual channels of the TCS3701 could enable colorimetric sensing using the built-in ALS.

Continuous monitoring using ALS — Continuous monitoring, such as tracking the UV index [65], is feasible; however, it would require the LED holder to continuously cover a small area of the screen, which is not ideal. Continuous sensing would be better suited for smartwatches with the ALS positioned alongside the display (Sony Smartwatch 3), ensuring the communication LED does not obstruct the screen.

Bi-directional Add-ons — The current version of Lightlink supports only input or output add-ons for the smartwatch, designed to explore low-fidelity extensions that complement its smartwatch’s high-fidelity capabilities. This approach minimizes complexity, cost, and power usage. Future versions could support bidirectional interaction by combining LED driving and photodiode decoding circuitry on a single PCB, with minor hardware changes and minimal software adjustments to integrate input and output. For example, a bidirectional add-on could use UV or air quality sensor data to show on an LED strap add-on.

Packet Loss — LightLink uses analog modulation for downlink communication to the Ambient Light Sensor, which eliminates packet loss by design. For uplink transmission, data is sent from the watch screen to external devices in 10-bit packets. Packet loss is minimized by selecting the most reliable transmission speed for each packet, as described in Section 7.2. In a conventional wireless system, packet loss can result from factors such as transmission distance, environmental interference, and signal power. In contrast, LightLink benefits from the close physical proximity of the transmitter and receiver. Moreover, optical interference is reduced through a custom 3D-printed holder that ensures precise alignment of the LED or photodiode.

9 CONCLUSION

This work explored augmenting smartwatches with sensor and actuator add-ons using visible light communication. Through a series of technical experiments, we characterized the smartwatch’s built-in ambient light sensor and screen as a light receiver and transmitter, respectively. The results from these experiments inform the design of hardware and communication strategies to establish a bi-directional link between the watch and the external add-ons. We validated our approach by demonstrating a suite of five sensor add-on devices (e.g. temperature sensor, color sensor) and three actuator add-ons (e.g. LED matrix). These add-on devices offer flexibility in placement around the watch, allowing for customized setup based on user needs. Additionally, we provide a toolkit framework with access to Arduino and Android libraries, enabling easy replication of the system and the development of custom add-ons tailored

to specific needs. We believe that using the built-in light sensor and screen of smart devices for communication with external devices can enable novel interactions that were previously difficult to achieve without significant hardware augmentations.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their valuable feedback. This work was made possible by the National Science and Engineering Research Council (NSERC) of Canada (RGPIN - 2023-03608), A-MEDICO (Alberta Medical Devices Innovation Consortium), the Alberta Innovates Postdoctoral Fellowship, and the University of Calgary. We also thank Anand Kumar for his assistance with video figure preparation.

REFERENCES

- [1] [n. d.]. AMS3701. <https://docs.rs-online.com/cd1e/A700000008290147.pdf>
- [2] Karan Ahuja, Sujeeth Pareddy, Robert Xiao, Mayank Goel, and Chris Harrison. 2019. LightAnchors: Appropriating Point Lights for Spatially-Anchored Augmented Reality Interfaces. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology* (New Orleans, LA, USA) (UIST '19). Association for Computing Machinery, New York, NY, USA, 189–196. <https://doi.org/10.1145/3332165.3347884>
- [3] Kamran Ali and Alex X. Liu. 2022. Fine-Grained Vibration Based Sensing Using a Smartphone. *IEEE Transactions on Mobile Computing* 21, 11 (2022), 3971–3985. <https://doi.org/10.1109/TMC.2021.3067679>
- [4] Justin Chan and Shyamnath Gollakota. 2022. Laser speckle using smartphone LiDAR. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services* (Portland, Oregon) (MobiSys '22). Association for Computing Machinery, New York, NY, USA, 632–633. <https://doi.org/10.1145/3498361.3538670>
- [5] Justin Chan, Ananditha Raghunath, Kelly E. Michaelsen, and Shyamnath Gollakota. 2022. Testing a Drop of Liquid Using Smartphone LiDAR. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 6, 1, Article 3 (March 2022), 27 pages. <https://doi.org/10.1145/3517256>
- [6] Liwei Chan, Stefanie Müller, Anne Roudaut, and Patrick Baudisch. 2012. CapStones and ZebraWidgets: sensing stacks of building blocks, dials and sliders on capacitive touch screens. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Austin, Texas, USA) (CHI '12). Association for Computing Machinery, New York, NY, USA, 2189–2192. <https://doi.org/10.1145/2207676.2208371>
- [7] Angie Chandler, Joe Finney, Carl Lewis, and Alan Dix. 2009. Toward emergent technology for blended public displays. In *Proceedings of the 11th International Conference on Ubiquitous Computing* (Orlando, Florida, USA) (UbiComp '09). Association for Computing Machinery, New York, NY, USA, 101–104. <https://doi.org/10.1145/1620545.1620562>
- [8] Tuochao Chen, Justin Chan, and Shyamnath Gollakota. 2022. Underwater messaging using mobile devices. In *Proceedings of the ACM SIGCOMM 2022 Conference* (SIGCOMM '22). ACM, 545–559. <https://doi.org/10.1145/3544216.3544258>
- [9] Xiang 'Anthony' Chen, Tovi Grossman, Daniel J. Wigdor, and George Fitzmaurice. 2014. Duet: exploring joint interactions on a smart phone and a smart watch. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Toronto, Ontario, Canada) (CHI '14). Association for Computing Machinery, New York, NY, USA, 159–168. <https://doi.org/10.1145/2556288.2556955>
- [10] Aritra Dhar, Aditya Nittala, and Kuldeep Yadav. 2016. TactBack: VibroTactile braille output using smartphone and smartwatch for visually impaired. In *Proceedings of the 13th International Web for All Conference* (Montreal, Canada) (W4A '16). Association for Computing Machinery, New York, NY, USA, Article 13, 2 pages. <https://doi.org/10.1145/2899475.2899514>
- [11] Christine Dierk, Molly Jane Pearce Nicholas, and Eric Paulos. 2018. AlterWear: Battery-Free Wearable Displays for Opportunistic Interactions. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/3173574.3173794>
- [12] Christine Dierk, Tomás Vega Gálvez, and Eric Paulos. 2017. AlterNail: Ambient, Batteryless, Stateful, Dynamic Displays at your Fingertips. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) (CHI '17). Association for Computing Machinery, New York, NY, USA, 6754–6759. <https://doi.org/10.1145/3025453.3025924>
- [13] Jun Gong, Da-Yuan Huang, Teddy Seyed, Te Lin, Tao Hou, Xin Liu, Molin Yang, Boyu Yang, Yuhang Zhang, and Xing-Dong Yang. 2018. Jetto: Using Lateral Force Feedback for Smartwatch Interactions. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–14. <https://doi.org/10.1145/3173574.3174000>
- [14] Jun Gong, Xing-Dong Yang, and Pourang Irani. 2016. WristWhirl: One-handed Continuous Smartwatch Input using Wrist Gestures. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology* (Tokyo, Japan) (UIST '16). Association for Computing Machinery, New York, NY, USA, 861–872. <https://doi.org/10.1145/2984511.2984563>
- [15] Saul Greenberg and Bill Buxton. 2008. Usability evaluation considered harmful (some of the time). In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Florence, Italy) (CHI '08). Association for Computing Machinery, New York, NY, USA, 111–120. <https://doi.org/10.1145/1357054.1357074>
- [16] Saul Greenberg and Chester Fitchett. 2001. Phidgets: easy development of physical interfaces through physical widgets. In *Proceedings of the 14th Annual ACM Symposium on User Interface Software and Technology* (Orlando, Florida) (UIST '01). Association for Computing Machinery, New York, NY, USA, 209–218. <https://doi.org/10.1145/502348.502388>
- [17] Steven Houben and Nicolai Marquardt. 2015. WatchConnect: A Toolkit for Prototyping Smartwatch-Centric Cross-Device Applications. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* (Seoul, Republic of Korea) (CHI '15). Association for Computing Machinery, New York, NY, USA, 1247–1256. <https://doi.org/10.1145/2702123.2702215>
- [18] Da-Yuan Huang, Ruizhen Guo, Jun Gong, Jingxian Wang, John Graham, De-Nian Yang, and Xing-Dong Yang. 2017. RetroShape: Leveraging Rear-Surface Shape Displays for 2.5D Interaction on Smartwatches. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology* (Québec City, QC, Canada) (UIST '17). Association for Computing Machinery, New York, NY, USA, 539–551. <https://doi.org/10.1145/3126594.3126610>
- [19] Michael Hung, Lora Oehlberg, and David Ledo. 2019. WatchPen: Using Cross-Device Interaction Concepts to Augment Pen-Based Interaction. In *Proceedings of the 21st International Conference on Human-Computer Interaction with Mobile Devices and Services* (Taipei, Taiwan) (MobileHCI '19). Association for Computing Machinery, New York, NY, USA, Article 63, 7 pages. <https://doi.org/10.1145/3338286.3344413>
- [20] Inhwon Hwang, Jungchan Cho, and Songhwai Oh. 2014. VibeComm: Radio-Free Wireless Communication for Smart Devices Using Vibration. *Sensors* 14, 11 (2014), 21151–21173. <https://doi.org/10.3390/s141121151>
- [21] Yong hyeon Kim and Yeon ho Chung. 2016. Experimental demonstration of light sensor-based visible light communications using time shift light intensity modulation. *Optical Engineering* 55, 9 (2016), 090501. <https://doi.org/10.1117/1.OE.55.9.090501>
- [22] Kaori Ikematsu and Kunihiro Kato. 2023. ShiftTouch: Extending Touchscreens with Passive Interfaces using Small Occluded Area for Discrete Touch Input. In *Proceedings of the Seventeenth International Conference on Tangible, Embedded, and Embodied Interaction* (Warsaw, Poland) (TEI '23). Association for Computing Machinery, New York, NY, USA, Article 11, 15 pages. <https://doi.org/10.1145/3569009.3572742>
- [23] Antony Albert Raj Irudayaraj, Jeremy Hartmann, Omid Abari, and Daniel Vogel. 2023. Scatterpixels: Ad Hoc Reconfigurable Physical Pixel Displays. In *Graphics Interface 2023*. <https://openreview.net/forum?id=t-IJQBTmDXV>
- [24] Saif Ismail and Muataz Salih. 2020. A review of visible light communication (VLC) technology. *AIP Conference Proceedings* 2213, 020289. <https://doi.org/10.1063/5.0000109>
- [25] Jakob Karolus, Francisco Kiss, Caroline Eckerth, Nicolas Viot, Felix Bachmann, Albrecht Schmidt, and Pawel W. Wozniak. 2021. EMBody: A Data-Centric Toolkit for EMG-Based Interface Prototyping and Experimentation. *Proc. ACM Hum.-Comput. Interact.* 5, EICS, Article 195 (May 2021), 29 pages. <https://doi.org/10.1145/3457142>
- [26] Rushil Khurana, Mayank Goel, and Kent Lyons. 2019. Detachable Smartwatch: More Than A Wearable. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 3, 2, Article 50 (June 2019), 14 pages. <https://doi.org/10.1145/3328921>
- [27] Hyunyoung Kim, Aluna Everitt, Carlos Tejada, Mengyu Zhong, and Daniel Ashbrook. 2021. MorpheusPlug: A Toolkit for Prototyping Shape-Changing Interfaces. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) (CHI '21). Association for Computing Machinery, New York, NY, USA, Article 101, 13 pages. <https://doi.org/10.1145/3411764.3445786>
- [28] Konstantin Klamka, Tom Horak, and Raimund Dachselt. 2020. Watch+Strap: Extending Smartwatches with Interactive StrapDisplays. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '20). Association for Computing Machinery, New York, NY, USA, 1–15. <https://doi.org/10.1145/3313831.3376199>
- [29] Sven Kratz, Tilo Westermann, Michael Rohs, and Georg Essl. 2011. CapWidgets: tangible widgets versus multi-touch controls on mobile devices. In *CHI '11 Extended Abstracts on Human Factors in Computing Systems* (Vancouver, BC, Canada) (CHI EA '11). Association for Computing Machinery, New York, NY, USA, 1351–1356. <https://doi.org/10.1145/1979742.1979773>
- [30] Gierad Laput and Chris Harrison. 2019. Sensing Fine-Grained Hand Activity with Smartwatches. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland UK) (CHI '19). Association for Computing

- Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3290605.3300568>
- [31] Gierad Laput, Robert Xiao, Xiang 'Anthony' Chen, Scott E. Hudson, and Chris Harrison. 2014. Skin buttons: cheap, small, low-powered and clickable fixed-icon laser projectors. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology* (Honolulu, Hawaii, USA) (UIST '14). Association for Computing Machinery, New York, NY, USA, 389–394. <https://doi.org/10.1145/2642918.2647356>
- [32] Gierad Laput, Robert Xiao, and Chris Harrison. 2016. ViBand: High-Fidelity Bio-Acoustic Sensing Using Commodity Smartwatch Accelerometers. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology* (Tokyo, Japan) (UIST '16). Association for Computing Machinery, New York, NY, USA, 321–333. <https://doi.org/10.1145/2984511.2984582>
- [33] David Ledo, Steven Houben, Jo Vermeulen, Nicolai Marquardt, Lora Oehlberg, and Saul Greenberg. 2018. Evaluation Strategies for HCI Toolkit Research. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–17. <https://doi.org/10.1145/3173574.3173610>
- [34] David Ledo, Jo Vermeulen, Sheelagh Carpendale, Saul Greenberg, Lora Oehlberg, and Sebastian Boring. 2019. Astral: Prototyping Mobile and Smart Object Interactive Behaviours Using Familiar Applications. In *Proceedings of the 2019 on Designing Interactive Systems Conference* (San Diego, CA, USA) (DIS '19). Association for Computing Machinery, New York, NY, USA, 711–724. <https://doi.org/10.1145/3322276.3322329>
- [35] Tianxing Li, Chuankai An, Andrew T. Campbell, and Xia Zhou. 2015. HiLight: Hiding Bits in Pixel Translucency Changes. *SIGMOBILE Mob. Comput. Commun. Rev.* 18, 3 (Jan. 2015), 62–70. <https://doi.org/10.1145/2721896.2721910>
- [36] Rong-Hao Liang, Kai-Yin Cheng, Liwei Chan, Chuan-Xhyuan Peng, Mike Y. Chen, Rung-Huei Liang, De-Nian Yang, and Bing-Yu Chen. 2013. GaussBits: magnetic tangible bits for portable and occlusion-free near-surface interactions. In *CHI '13 Extended Abstracts on Human Factors in Computing Systems* (Paris, France) (CHI EA '13). Association for Computing Machinery, New York, NY, USA, 2837–2838. <https://doi.org/10.1145/2468356.2479537>
- [37] Rong-Hao Liang, Han-Chih Kuo, Liwei Chan, De-Nian Yang, and Bing-Yu Chen. 2014. GaussStones: shielded magnetic tangibles for multi-token interactions on portable displays. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology* (Honolulu, Hawaii, USA) (UIST '14). Association for Computing Machinery, New York, NY, USA, 365–372. <https://doi.org/10.1145/2642918.2647384>
- [38] Xiangyu Liu, Xuetao Wei, Lei Guo, and Song Song. 2022. DarkVLP: "Lights-Off" Visible-Light Positioning. *IEEE Internet of Things Journal* 9, 13 (2022), 11071–11084. <https://doi.org/10.1109/JIOT.2021.3125795>
- [39] Jess McIntosh, Paul Strohmeier, Jarrod Knibbe, Sebastian Boring, and Kasper Hornbæk. 2019. Magnetips: Combining Fingertip Tracking and Haptic Feedback for Around-Device Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland UK) (CHI '19). Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3290605.3300638>
- [40] Manuel Meier, Paul Strel, Andreas Rene Fender, and Christian Holz. 2021. Demonstrating TapID for Rapid Touch Interaction on Surfaces in Virtual Reality for Productivity Scenarios. In *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) (CHI EA '21). Association for Computing Machinery, New York, NY, USA, Article 175, 4 pages. <https://doi.org/10.1145/3411763.3451553>
- [41] Rajalakshmi Nandakumar, Krishna Kant Chintalapudi, Venkat Padmanabhan, and Ramarathnam Venkatesan. 2013. Dhvani: secure peer-to-peer acoustic NFC. *SIGCOMM Comput. Commun. Rev.* 43, 4 (Aug. 2013), 63–74. <https://doi.org/10.1145/2534169.2486037>
- [42] Aditya Shekhar Nittala, Xing-Dong Yang, Scott Bateman, Ehud Sharlin, and Saul Greenberg. 2015. PhoneEar: interactions for mobile devices that hear high-frequency sound-encoded data. In *Proceedings of the 7th ACM SIGCHI Symposium on Engineering Interactive Computing Systems* (Duisburg, Germany) (EICS '15). Association for Computing Machinery, New York, NY, USA, 174–179. <https://doi.org/10.1145/2774225.2775082>
- [43] Simon Olberding, Michael Wessely, and Jürgen Steimle. 2014. PrintScreen: fabricating highly customizable thin-film touch-displays. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology* (Honolulu, Hawaii, USA) (UIST '14). Association for Computing Machinery, New York, NY, USA, 281–290. <https://doi.org/10.1145/2642918.2647413>
- [44] Dan R. Olsen. 2007. Evaluating user interface systems research. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology* (Newport, Rhode Island, USA) (UIST '07). Association for Computing Machinery, New York, NY, USA, 251–258. <https://doi.org/10.1145/1294211.1294256>
- [45] Henning Pohl, Justyna Medrek, and Michael Rohs. 2016. ScatterWatch: subtle notifications via indirect illumination scattered in the skin. In *Proceedings of the 18th International Conference on Human-Computer Interaction with Mobile Devices and Services* (Florence, Italy) (MobileHCI '16). Association for Computing Machinery, New York, NY, USA, 7–16. <https://doi.org/10.1145/2935334.2935351>
- [46] Dirk Queck, Iannis Albert, Nicole Burkard, Philipp Zimmer, Georg Volkmar, Bastian Dänekas, Rainer Malaka, and Marc Herrlich. 2022. SpiderClip: Towards an Open Source System for Wearable Device Simulation in Virtual Reality. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) (CHI EA '22). Association for Computing Machinery, New York, NY, USA, Article 341, 7 pages. <https://doi.org/10.1145/3491101.3519758>
- [47] Gabriel Reyes, Dingtian Zhang, Sarthak Ghosh, Pratik Shah, Jason Wu, Aman Parnami, Bailey Bercik, Thad Starner, Gregory D. Abowd, and W. Keith Edwards. 2016. Whoosh: non-voice acoustics for low-cost, hands-free, and rapid input on smartwatches. In *Proceedings of the 2016 ACM International Symposium on Wearable Computers* (Heidelberg, Germany) (ISWC '16). Association for Computing Machinery, New York, NY, USA, 120–127. <https://doi.org/10.1145/2971763.2971765>
- [48] Nirupam Roy, Mahanth Gowda, and Romit Roy Choudhury. 2015. Ripple: communicating through physical vibration. In *Proceedings of the 12th USENIX Conference on Networked Systems Design and Implementation* (Oakland, CA) (NSDI'15). USENIX Association, USA, 265–278.
- [49] Sutirtha Roy, Moshfiq-U-Saleheen Chowdhury, Jurjaan Onayza Noim, Richa Pandey, and Aditya Shekhar Nittala. 2024. HoloChemie - Sustainable Fabrication of Soft Biochemical Holographic Devices for Ubiquitous Sensing. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (Pittsburgh, PA, USA) (UIST '24). Association for Computing Machinery, New York, NY, USA, Article 111, 19 pages. <https://doi.org/10.1145/3654777.3676448>
- [50] Munehiko Sato, Atsushi Hiyama, Tomohiro Tanikawa, and Michitaka Hirose. 2009. Particle Display System — Virtually Perceivable Pixels with Randomly Distributed Physical Pixels —. *Journal of Information Processing* 17 (dec 2009), 280–291. <https://doi.org/10.2197/ipsjip.17.280>
- [51] Léa Saviot, Frederik Brudy, and Steven Houben. 2017. WRISTBAND.IO: Expanding Input and Output Spaces of a Smartwatch. In *Proceedings of the 2017 CHI Conference Extended Abstracts on Human Factors in Computing Systems* (Denver, Colorado, USA) (CHI EA '17). Association for Computing Machinery, New York, NY, USA, 2025–2033. <https://doi.org/10.1145/3027063.3053132>
- [52] Martin Schmitz, Martin Herbers, Niloofar Dezfuli, Sebastian Günther, and Max Mühlhäuser. 2018. Off-Line Sensing: Memorizing Interactions in Passive 3D-Printed Objects. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–8. <https://doi.org/10.1145/3173574.3173756>
- [53] Martin Schmitz, Mohammadreza Khalilbeigi, Matthias Balwierz, Roman Lissermann, Max Mühlhäuser, and Jürgen Steimle. 2015. Capricate: A Fabrication Pipeline to Design and 3D Print Capacitive Touch Sensors for Interactive Objects. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology* (Charlotte, NC, USA) (UIST '15). Association for Computing Machinery, New York, NY, USA, 253–258. <https://doi.org/10.1145/2807442.2807503>
- [54] Martin Schmitz, Jürgen Steimle, Jochen Huber, Niloofar Dezfuli, and Max Mühlhäuser. 2017. Flexibles: Deformation-Aware 3D-Printed Tangibles for Capacitive Touchscreens. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) (CHI '17). Association for Computing Machinery, New York, NY, USA, 1001–1014. <https://doi.org/10.1145/3025453.3025663>
- [55] Julia Schwarz, David Klionsky, Chris Harrison, Paul Dietz, and Andrew Wilson. 2012. Phone as a pixel: enabling ad-hoc, large-scale displays using mobile devices. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Austin, Texas, USA) (CHI '12). Association for Computing Machinery, New York, NY, USA, 2235–2238. <https://doi.org/10.1145/2207676.2208378>
- [56] Teddy Seyed, Alaa Azazi, Edwin Chan, Yuxi Wang, and Frank Maurer. 2015. SoD-Toolkit: A Toolkit for Interactively Prototyping and Developing Multi-Sensor, Multi-Device Environments. In *Proceedings of the 2015 International Conference on Interactive Tabletops & Surfaces* (Madeira, Portugal) (ITS '15). Association for Computing Machinery, New York, NY, USA, 171–180. <https://doi.org/10.1145/2817721.2817750>
- [57] Joyce Siqueira, Fabrizio Alphonso Alves de Melo Soares, Deller James Ferreira, Cleyton Rafael Gomes Silva, Luciana de Oliveira Berretta, Cristiane Bastos Rocha Ferreira, Igor Moreira Félix, Anderson da Silva Soares, Ronaldo Martins da Costa, and Mateus Machado Luna. 2016. Braille Text Entry on Smartphones: A Systematic Review of the Literature. In *2016 IEEE 40th Annual Computer Software and Applications Conference (COMPSAC)*, Vol. 2. 521–526. <https://doi.org/10.1109/COMPSAC.2016.74>
- [58] Srinath Sridhar, Anders Markussen, Antti Oulasvirta, Christian Theobalt, and Sebastian Boring. 2017. WatchSense: On- and Above-Skin Input Sensing through a Wearable Depth Sensor. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) (CHI '17). Association for Computing Machinery, New York, NY, USA, 3891–3902. <https://doi.org/10.1145/3025453.3026005>
- [59] Matthew Stafford, Adriana Rogers, Shela Wu, Charles Carver, N. Sertac Artan, and Ziqian Dong. 2017. TETRIS: Smartphone-to-Smartphone Screen-Based Visible Light Communication. In *2017 IEEE 14th International Conference on Mobile Ad Hoc and Sensor Systems (MASS)*. 570–574. <https://doi.org/10.1109/MASS.2017.101>

- [60] Dennis Stanke, Benjamin Simon, Sergej Löwen, and Michael Rohs. 2024. Case-Touch: Occlusion-Free Touch Input by adding a Thin Sensor Stripe to the Smartwatch Case. In *Proceedings of the International Conference on Mobile and Ubiquitous Multimedia (MUM '24)*. Association for Computing Machinery, New York, NY, USA, 172–183. <https://doi.org/10.1145/3701571.3701583>
- [61] Ahren Studer, Timothy Passaro, and Lujo Bauer. 2011. Don't Bump, Shake on It: the exploitation of a popular accelerometer-based smart phone exchange and its secure replacement. In *Proceedings of the 27th Annual Computer Security Applications Conference (Orlando, Florida, USA) (ACSAC '11)*. Association for Computing Machinery, New York, NY, USA, 333–342. <https://doi.org/10.1145/2076732.2076780>
- [62] Akifumi Takahashi, Yudai Tanaka, Archit Tamhane, Alan Shen, Shan-Yuan Teng, and Pedro Lopes. 2024. Can a Smartwatch Move Your Fingers? Compact and Practical Electrical Muscle Stimulation in a Smartwatch. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (Pittsburgh, PA, USA) (UIST '24)*. Association for Computing Machinery, New York, NY, USA, Article 2, 15 pages. <https://doi.org/10.1145/3654777.3676373>
- [63] Zhao Tian, Kevin Wright, and Xia Zhou. 2016. The darkLight rises: visible light communication in the dark. In *Proceedings of the 22nd Annual International Conference on Mobile Computing and Networking (New York City, New York) (MobiCom '16)*. Association for Computing Machinery, New York, NY, USA, 2–15. <https://doi.org/10.1145/2973750.2973772>
- [64] Riku Tsunoda, Myunguen Choi, and Buntarou Shizuki. 2024. Thumb-to-Finger Gesture Recognition Using COTS Smartwatch Accelerometers. In *Proceedings of the International Conference on Mobile and Ubiquitous Multimedia (MUM '24)*. Association for Computing Machinery, New York, NY, USA, 184–195. <https://doi.org/10.1145/3701571.3701600>
- [65] Anandghan Waghmare, Ishan Chatterjee, Vikram Iyer, and Shwetak Patel. 2024. WatchLink: Enhancing Smartwatches with Sensor Add-Ons via ECG Interface. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (Pittsburgh, PA, USA) (UIST '24)*. Association for Computing Machinery, New York, NY, USA, Article 14, 13 pages. <https://doi.org/10.1145/3654777.3676329>
- [66] Anandghan Waghmare, Farshid Salemi Parizi, Jason Hoffman, Yuntao Wang, Matthew Thompson, and Shwetak Patel. 2023. GlucoScreen: A Smartphone-based Readerless Glucose Test Strip for Prediabetes Screening. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 7, 1, Article 30 (March 2023), 20 pages. <https://doi.org/10.1145/3580855>
- [67] Zeyu Wang, Zhice Yang, Qianyi Huang, Lin Yang, and Qian Zhang. 2019. ALS-P: Light Weight Visible Light Positioning via Ambient Light Sensor. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*. 1306–1314. <https://doi.org/10.1109/INFOCOM.2019.8737575>
- [68] Dirk Wenig, Johannes Schöning, Alex Olwal, Mathias Oben, and Rainer Malaka. 2017. WatchThru: Expanding Smartwatch Displays with Mid-air Visuals and Wrist-worn Augmented Reality. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems (Denver, Colorado, USA) (CHI '17)*. Association for Computing Machinery, New York, NY, USA, 716–721. <https://doi.org/10.1145/3025453.3025852>
- [69] Robert Xiao, Teng Cao, Ning Guo, Jun Zhuo, Yang Zhang, and Chris Harrison. 2018. LumiWatch: On-Arm Projected Graphics and Touch Input. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (Montreal QC, Canada) (CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/3173574.3173669>
- [70] Robert Xiao, Gierad Laput, and Chris Harrison. 2014. Expanding the input expressivity of smartwatches with mechanical pan, twist, tilt and click. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (Toronto, Ontario, Canada) (CHI '14)*. Association for Computing Machinery, New York, NY, USA, 193–196. <https://doi.org/10.1145/2556288.2557017>
- [71] Robert Xiao, Sven Mayer, and Chris Harrison. 2020. VibroComm: Using Commodity Gyroscopes for Vibroacoustic Data Reception. In *22nd International Conference on Human-Computer Interaction with Mobile Devices and Services (Oldenburg, Germany) (MobileHCI '20)*. Association for Computing Machinery, New York, NY, USA, Article 5, 9 pages. <https://doi.org/10.1145/3379503.3403540>
- [72] Jackie (Junrui) Yang and James A. Landay. 2019. InfoLED: Augmenting LED Indicator Lights for Device Positioning and Communication. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (New Orleans, LA, USA) (UIST '19)*. Association for Computing Machinery, New York, NY, USA, 175–187. <https://doi.org/10.1145/3332165.3347954>
- [73] Chungkuk Yoo, Inseok Hwang, Seungwoo Kang, Myung-Chul Kim, Seonghoon Kim, Daeyoung Won, Yu Gu, and Junehwa Song. 2017. Card-stunt as a Service: Empowering a Massively Packed Crowd for Instant Collective Expressiveness. In *Proceedings of the 15th Annual International Conference on Mobile Systems, Applications, and Services (Niagara Falls, New York, USA) (MobiSys '17)*. Association for Computing Machinery, New York, NY, USA, 121–135. <https://doi.org/10.1145/3081333.3081357>
- [74] Yu Zeng, Yunqing Chen, Shuntian Feng, Kaiyu Zhou, Xianghui Sun, Tong Mao, Lei Shi, Baohua Lei, Feng Wang, Xiaojun Yan, Quanhui Xu, and Bofei Zhang. 2014. Visible light communication system for mobile device. In *2014 Sixth International Conference on Ubiquitous and Future Networks (ICUFN)*. 26–28. <https://doi.org/10.1109/ICUFN.2014.6876742>
- [75] Yang Zhang and Chris Harrison. 2015. Tomo: Wearable, Low-Cost Electrical Impedance Tomography for Hand Gesture Recognition. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology (Charlotte, NC, USA) (UIST '15)*. Association for Computing Machinery, New York, NY, USA, 167–173. <https://doi.org/10.1145/2807442.2807480>
- [76] Yang Zhang, Junhan Zhou, Gierad Laput, and Chris Harrison. 2016. SkinTrack: Using the Body as an Electrical Waveguide for Continuous Finger Tracking on the Skin. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems (San Jose, California, USA) (CHI '16)*. Association for Computing Machinery, New York, NY, USA, 1491–1503. <https://doi.org/10.1145/2858036.2858082>
- [77] Yi Zhao, Joshua R. Smith, and Alanson Sample. 2015. NFC-WISP: an open source software defined near field RFID sensing platform. In *Adjunct Proceedings of the 2015 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2015 ACM International Symposium on Wearable Computers (Osaka, Japan) (UbiComp/ISWC'15 Adjunct)*. Association for Computing Machinery, New York, NY, USA, 369–372. <https://doi.org/10.1145/2800835.2800912>
- [78] Junhan Zhou, Yang Zhang, Gierad Laput, and Chris Harrison. 2016. AuraSense: Enabling Expressive Around-Smartwatch Interactions with Electric Field Sensing. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology (Tokyo, Japan) (UIST '16)*. Association for Computing Machinery, New York, NY, USA, 81–86. <https://doi.org/10.1145/2984511.2984568>